



Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение высшего
образования «Московский государственный технический университет
имени Н.Э. Баумана (национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ Робототехника и комплексная автоматизация

КАФЕДРА Системы автоматизированного проектирования (РК-6)

РАСЧЕТНО-ПОЯСНИТЕЛЬНАЯ ЗАПИСКА

К ВЫПУСКНОЙ КВАЛИФИКАЦИОННОЙ РАБОТЕ

НА ТЕМУ

«Разработка инструментов для создания видеоигр в жанре
Top-Down RPG на Unreal Engine 5»

Студент РК6-41М
(Группа)

А.В. Фёдоров
(подпись, дата) (инициалы и фамилия)

Руководитель ВКР

Ф.А. Витюков
(подпись, дата) (инициалы и фамилия)

Нормоконтролер

С.В. Грошев
(подпись, дата) (инициалы и фамилия)

Москва, 2026 г.

УТВЕРЖДАЮ

Заведующий кафедрой РК-6
(индекс)

_____ А.П. Карпенко
(инициалы и фамилия)

«___» _____ 2026 г.

ЗАДАНИЕ

на выполнение выпускной квалификационной работы

Студент группы РК6-41М

_____ Фёдоров Артемий Владиславович
(фамилия, имя, отчество)

Тема выпускной квалификационной работы

Разработка инструментов для создания видеоигр в жанре Top-Down RPG на Unreal Engine 5

При выполнении ВКР:

	Используется / Не используется	Да/Нет
1)	Литературные источники и документы, имеющие гриф секретности	Нет
2)	Литературные источники и документы, имеющие пометку «Для служебного пользования», иных пометок, запрещающих открытое опубликование	Нет
3)	Служебные материалы других организаций	Нет
4)	Результаты НИР (ОКР), выполняемой в МГТУ им. Н.Э. Баумана	Нет
5)	Материалы по незавершенным исследованиям или материалы по завершенным исследованиям, но ещё не опубликованные в открытой печати	Нет

Тема выпускной квалификационной работы утверждена распоряжением по факультету:	
Название факультета:	«Робототехника и комплексная автоматизация»
Дата и рег. номер распоряжения:	№ 03.07 – 03/99 от «18» декабря 2025 г.

Часть 1. Аналитическая

Провести анализ существующих решений для создания тайловых игровых полей в игровых движках. Провести визуальный анализ и анализ пользовательского интерфейса коммерчески успешных проектов Top Down TPG разных лет. Изучить архитектуру плагинов для Unreal Engine 5, включая механизмы расширения редактора через Editor Mode и интерактивные инструменты. Изучить средства разработки пользовательских интерфейсов в Unreal Engine 5.

Часть 2. Практическая часть 1. Разработка инструментов для создания карт, разбитых на клетки

Разработать плагин для Unreal Engine 5, расширяющий редактор инструментами для создания тайловых игровых полей. Создать специальный Editor Mode для работы с клетками. Разработать набор интерактивных инструментов для рисования и удаления тайлов, а

также для размещения объектов на игровом поле. Реализовать систему визуальной обратной связи, отображающую типы клеток и границы сетки.

Часть 3. Практическая часть 2. Разработка игровых персонажей

Разработать управляемого игрового персонажа для Top-Down RPG. Реализовать систему навигации по разработанной тайловой сеткой. Разработать систему способностей на базе, позволяющую создавать действия, применяемые на игровое поле.

Часть 4. Практическая часть 3. Разработка пользовательского интерфейса

Разработать камеру, позволяющую осматривать игровое поле сверху. Разработать систему пользовательского интерфейса для выбора персонажей. Создать интерфейс персонажа, отображающий набор его способностей с использованием UMG. Обеспечить интеграцию интерфейсных элементов с игровой логикой.

Оформление выпускной квалификационной работы:

Расчетно-пояснительная записка на 74 листах формата А4.

Перечень графического (иллюстративного) материала (чертежи, плакаты, слайды и т.п.):

10 слайдов

Дата выдачи задания «10» февраля 2026 г.

Студент

А.В. Фёдоров

(Подпись, дата)

(И.О. Фамилия)

Руководитель выпускной квалификационной работы

Ф.А. Витюков

(Подпись, дата)

(И.О. Фамилия)

Примечание: Задание оформляется в двух экземплярах: один выдается студенту, второй хранится на кафедре.

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение высшего образования
«Московский государственный технический университет имени Н.Э. Баумана (национальный
исследовательский университет)» (МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ РК
КАФЕДРА РК-6
ГРУППА РК6-41М

УТВЕРЖДАЮ
Заведующий кафедрой РК-6
(индекс)

_____ А.П. Карпенко
(инициалы и фамилия)

« ____ » _____ 2026 г.

КАЛЕНДАРНЫЙ ПЛАН
выполнения выпускной квалификационной работы

студента: Фёдоров Артемий Владиславович
(Фамилия, имя, отчество)

Тема выпускной квалификационной работы: Разработка инструментов для создания видео-игр в жанре Top-Down RPG на Unreal Engine 5

№ п/п	Наименование этапов выпускной квалификационной работы	Сроки выполнения этапов		Отметка о выполнении	
		план	факт	Должность	ФИО, подпись
1.	Задание на выполнение работы. Формулирование проблемы, цели и задач работы	10.02.2026	_____	Руководитель ВКР	Витюков Ф.А.
2.	Часть 1: Аналитическая	20.02.2026	_____	Руководитель ВКР	Витюков Ф.А.
3.	Утверждение окончательных формулировок решаемой проблемы, цели работы и задачи	27.02.2026	_____	Заведующий кафедрой	Карпенко А.П.
4.	Часть 2: Практическая часть 1.	07.04.2026	_____	Руководитель ВКР	Витюков Ф.А.
5.	Часть 3: Практическая часть 2	21.04.2026	_____	Руководитель ВКР	Витюков Ф.А.
6.	Часть 4: Практическая часть 3	22.05.2026	_____	Руководитель ВКР	Витюков Ф.А.
7.	1-я редакция работы	25.05.2026	_____	Руководитель ВКР	Витюков Ф.А.
8.	Подготовка доклада и презентации	07.07.2026	_____	Руководитель ВКР	Витюков Ф.А.
9.	Заключение руководителя	07.06.2026	_____	Руководитель ВКР	Витюков Ф.А.
10.	Допуск работы к защите на ГЭК (нормоконтроль)	05.06.2026	_____	Нормоконтролер	Грошев С.В.
11.	Внешняя рецензия	07.06.2026	_____	Секретарь ГЭК	Балыкина А.М.
12.	Защита работы на ГЭК	10.06.2026	_____	Секретарь ГЭК	Балыкина А.М.

Студент _____ А.В. Фёдоров
(подпись, дата)

Руководитель работы _____ Ф.А. Витюков
(подпись, дата)

АННОТАЦИЯ

Данная работа посвящена разработке комплекса инструментов для создания игр в жанре Top-Down RPG на платформе Unreal Engine 5.

В рамках исследования решены следующие задачи: анализ визуальной составляющей и UI успешных игровых проектов в жанре Top-Down RPG; разработка инструмента для создания игровых полей, разделенных на ячейки; разработка инструментов для размещения персонажей и объектов на игровом поле и настройка навигации персонажей по игровому полю; разработка инструмента для создания построек произвольной формы; разработка базового UI и системы взаимодействия, позволяющей пользователю взаимодействовать с элементами игрового мира.

Тип работы: выпускная квалификационная работа.

Тема работы: “Разработка инструментов для создания видеоигр в жанре Top-Down RPG на Unreal Engine 5”

Объект исследований: Процесс разработки плагина, расширяющего функционал игрового движка Unreal Engine 5 для возможности создания проектов Top-Down RPG.

СОДЕРЖАНИЕ

АННОТАЦИЯ	5
ВВЕДЕНИЕ.....	11
1 Исследование существующих проектов	16
1.1 Визуальный анализ.....	16
1.2 Анализ пользовательского интерфейса.....	20
2 Обзор используемых технологий	22
3 Разработка игрового поля	24
3.1 Хранение в памяти.....	27
3.2 Навигация.....	29
3.3 Оптимизация отображения сетки	29
4 Разработка режимов редактора	31
4.1 Настройка статусов ячеек.....	31
4.2 Размещение объектов на сетке	32
5 Разработка игрового персонажа	36
5.1 Разработка системы действий персонажа.....	37
6 Разработка пользовательского интерфейса	39
6.1 Управление камерой	39
6.2 Система состояний игрока	42
6.3 Разработка виджетов.....	45
6.4 Связывание пользовательского интерфейса с разработанными игровыми механиками.....	47
7 Применение разработанных инструментов.....	50
8 Анализ результатов	55
ЗАКЛЮЧЕНИЕ	57
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	58
ПРИЛОЖЕНИЕ А. Графическое представление выпускной квалификационной работы.....	62

ОПРЕДЕЛЕНИЯ, ОБОЗНАЧЕНИЯ И СОКРАЩЕНИЯ

Game Engine (игровой движок) - набор ключевых компонентов программного обеспечения, используемых для разработки игр и иных 3d-приложений. Как правило, инструменты движка абстрагированы от специфики конкретной игры, но могут учитывать некоторые особенности жанра - они предоставляют «базис» для разработки, «надстройку» над которым создает его пользователь.

Unreal Engine 5 (UE5) - игровой движок, разрабатываемый и поддерживаемый компанией Epic Games.

Open Source - модель разработки программного обеспечения, при которой исходный код программы открыт для изучения, модификации и свободного распространения.

Top-Down RPG - жанр компьютерных ролевых игр, в котором камера расположена сверху и смотрит вниз, обеспечивая обзор игрового поля с высоты.

Scene (сцена) - контейнер, содержащий все объекты игрового мира в текущий момент времени, включая персонажей, окружение, источники света, камеры.

Editor (редактор) - интегрированная среда разработки (IDE) игрового движка Unreal Engine, предоставляющая визуальные инструменты для создания, редактирования и отладки уровней, ассетов, материалов, анимаций, пользовательских интерфейсов и игровой логики.

Blueprints (Блюпринт) - визуальная система скриптинга Unreal Engine, позволяющая создавать игровую логику без написания кода.

Tilemap (сетка) - способ организации игрового пространства в виде регулярной сетки, разбитой на дискретные ячейки (тайлы или клетки).

Tile (тайл, клетка) - элементарная ячейка игрового поля в тайловой сетке. Может иметь различные статусы и визуальное представление.

Asset (ассет) - в контексте компьютерных игр, объект, представляющий собой единицу контента. Игровыми ассетами являются 3d-модели, текстуры, материалы, аудиофайлы, и т.д. Как правило, созданием ассетов занимаются художники, дизайнеры, музыканты.

Game Design (Гейм-дизайн) - дисциплина, занимающаяся проектированием игрового процесса, определением правил, целей, механик и способов взаимодействия игрока с игровым миром.

Game Feel - совокупность тактильных и сенсорных ощущений игрока при взаимодействии с игрой, достигаемая за счёт различных художественных приёмов.

Actions / Abilities (игровые действия / способности) - сущности, описывающие действия, которые игровой персонаж может совершать в игровом мире, например атаки, заклинания, перемещение и использование предметов.

Player Controller - компонент игрового движка, отвечающий за связь между действиями игрока (ввод с клавиатуры, мыши, геймпада) и поведением управляемого персонажа в игровом мире.

NavMesh (навигационная сетка) - представление доступного для перемещения пространства в виде множества выпуклых многоугольников, используемое системами искусственного интеллекта для поиска пути.

HUD (heads-up display) - совокупность элементов пользовательского интерфейса, отображаемых поверх игрового мира.

Actor (актёр) - в рамках движка UE5 любой объект, который может быть размещен на уровне. Базовый класс всех actor'ов - AActor.

Actor Component - специальный тип объекта, который может быть присоединен к выбранному actor'у как подобъект (subobject). Как правило, используется для внедрения функциональности, общей для различных actor'ов. Базовый класс - UActorComponent.

Transform (трансформ) - данные о местоположении, повороте и масштабе объекта. Представляются матрицей преобразований.

Scene Component (USceneComponent) - класс, производный от UActorComponent. Представляет собой компонент, который может иметь свой трансформ на сцене. Данный компонент используется для внедрения функциональности, не требующей геометрического представления.

Static Mesh (UStaticMesh) - класс, представляющий собой статический геометрический объект. Хранит данные о полигональной сетке модели.

Static Mesh Component (UStaticMeshComponent) - компонент, который может иметь статический меш и набор материалов, применимых к нему.

Geometry instancing (инстансинг, дублирование геометрии) - техника, позволяющая отрисовывать множество однотипных элементов за один проход.

Instanced Static Mesh Component (UInstancedStaticMeshComponent, ISMC) - класс, производный от UStaticMeshComponent, позволяющий использовать механизм инстансинга геометрии.

Runtime Virtual Texture (RVT) - технология Unreal Engine, позволяющая записывать данные сцены (цвет, высоту, нормали) в текстуру в реальном времени и использовать их в шейдерах других объектов. Применяется для проецирования сетки на ландшафт.

3d-model (3d-модель) - математическое представление объекта в трехмерном пространстве.

3d-modeling (3d-моделирование) - процесс создания 3d-модели объекта.

Текстурирование - процесс создания текстур объекта.

Rendering (рендеринг, отрисовка) - процесс получения 2d-изображения по имеющейся 3d-модели.

Текстура - изображение, накладываемое на поверхность 3d-модели. Может содержать одно или несколько свойств поверхности, например: цвет, жёсткость (roughness), смещение (displacement), направление нормалей (normal map), и т.д.

Shader (Шейдер) - компьютерная программа, выполняющаяся параллельно на графическом процессоре, и служащая для различных графических вычислений, таких как отрисовка 3d моделей, расчёт освещения и так далее.

Материал - набор свойств, определяющих поведение света при отражении от поверхности. Материалы также могут использовать одну или несколько текстур.

Material (UMaterial) - класс, позволяющий хранить и модифицировать информацию о материале.

ЯП - Язык программирования.

ВВЕДЕНИЕ

Видеоигры являются одной из наиболее динамично развивающихся и финансово успешных отраслей современной индустрии развлечений. Жанр *Top-Down RPG* (ролевая игра с видом сверху) относится к числу старейших в игровой индустрии, однако, продолжает пользоваться значительной популярностью у широкой аудитории. Современные проекты, такие как *Baldur's Gate 3* студии *Larian Studios* и *XCOM 2* компании *2K Games*, получают множество наград и имеют миллионы проданных копий, что демонстрирует коммерческий потенциал жанра.

Создание качественной игры в данном жанре сопряжено с рядом технических сложностей: от разработки визуальной составляющей и качественных игровых механик до удобного пользовательского интерфейса и эффективной системы управления. Данная работа фокусируется на ключевых аспектах, необходимых для разработки *Top-Down RPG*, а конкретно её поджанра *Tile-Based Top-Down RPG*, определяющегося наличием игровых карт, поделенными на клетки.

На рынке представлен ряд решений, подходящих под задачи разработки инструментов для *Top-Down RPG*:

- *RPG Maker (Kadokawa Games)*. Это одно из наиболее известных решений для создания ролевых игр с видом сверху. Инструмент предоставляет редактор карт, разбитых на тайлы; событийную систему для описания поведения NPC; базу данных для управления персонажами, навыками и предметами. Основным ограничением является жесткая привязка к 2D-графике и отсутствие гибкости для создания нестандартных игровых механик без использования сложных скриптов на ЯП Ruby [1].
- *Turn Based Strategy Framework (Unity)*. Это коммерческий фреймворк для Unity, покрывающий все базовые системы пошаговых стратегий. Включает поддержку шестиугольных и квадратных сеток, системы ходов, управление действиями юнитов, базовый ИИ и поддержку многопользовательской игры. Недостатком для целей настоящей работы является ориентация на игровой движок Unity. Unity не является open-source движком

вследствие чего гораздо менее открыт для глубокой кастомизации в отличие от Unreal Engine [2].

- *Unity Tilemap System*. Встроенная в Unity система для создания 2D-уровней на основе тайлов. Предоставляет инструменты для "рисования" тайлов, слои, анимацию тайлов и правила для автоматического размещения углов и стыков (rule tiles). Данная система аналогично предыдущей имеет недостаток в жесткой привязке к фреймворку Unity [3].
- *Advanced Turn Based Tile Toolkit Plugin* для Unreal Engine. Это коммерческий плагин, разработанный *Knut Øverbye* и доступный в официальном магазине Epic Games Marketplace. Плагин позиционируется как полнофункциональное решение для создания пошаговых тактических и стратегических игр с использованием тайловых сеток. Он реализует большую часть функционала, рассмотренного в этой работе, однако имеет серьёзный недостаток в виде реализации на чистом Blueprints. Графический ЯП Blueprints может рассматриваться как преимущество для работы художников и гейм-дизайнеров, однако имеет критические недостатки при расширении проекта: отсутствие возможности реализации высокопроизводительных вычислений на C++; проблемы с производительностью при обработке больших объёмов данных (например поиск пути для юнитов с большим радиусом перемещения); отсутствие чёткого разделения на модули и интерфейсы, что усложняет интеграцию с другими системами. [4]

Анализ четырех существующих аналогов даёт обоснование целесообразности создания собственного решения.

В данной работе рассматривается создание и разработка элементов плагина для игрового движка Unreal Engine 5, предназначенного для разработки проектов в жанре Top-Down RPG. Работа делится на три основные части:

- Разработка инструментов для создания игровых карт, разбитых на клетки;
- Разработка игровых персонажей;
- Разработка пользовательского интерфейса.

Часть первая. Разработка инструментов для создания игровых карт, разбитых на клетки.

Центральным элементом большинства проектов в жанре Top-Down RPG является игровое поле, организованное в виде регулярной сетки (grid). Разбиение игрового пространства на дискретные ячейки - клетки (тайлы) - позволяет унифицировать логику перемещения персонажей, определения области действия способностей, расчёта линий обзора и построения маршрутов. Такой подход широко применяется как в классических представителях жанра (Fallout 1 и 2, Arcanum), так и в современных реализациях (XCOM 2, Baldur's Gate 3).

Для обеспечения возможности создания таких игровых полей непосредственно в среде разработки необходимо расширение штатного функционала редактора Unreal Engine 5 [5]. В рамках первой части работы рассмотрена разработка набора инструментов для создания и редактирования тайловых игровых карт, включающего:

- редакторский режим (Editor Mode) для работы с клетками [6];
- интерактивные инструменты рисования и удаления тайлов;
- интерактивные инструменты помещения игровых объектов на тайлы [7];
- систему визуальной отладки игрового поля.

Основная задача разрабатываемого редакторского режима и его инструментов - предоставить разработчику возможность интуитивного взаимодействия с тайловой сеткой непосредственно через окно просмотра уровня (viewport). Дополнительной задачей является реализация системы визуальной обратной связи, отображающей границы клеток и их статусы (проходимые, непроходимые, специальные).

Часть вторая. Разработка игровых персонажей.

Игровые персонажи являются неотъемлемой частью любой ролевой игры. В контексте Top-Down RPG персонажи характеризуются набором атрибутов (характеристики, навыки, состояние здоровья), поведением (перемещение, атака,

взаимодействие с окружением) и визуальным представлением (3D-модели, анимации).

В рамках второй части работы рассмотрена разработка следующих компонентов игровых персонажей:

- система навигации персонажа по игровому полю;
- система способностей персонажа.

Система автоматической навигации по полю критически важна для жанра Top-Down RPG, где задача игрока заключается в решении игровых ситуаций путём отдачи команд игровым персонажам. Игрок должен думать о результатах действий персонажа, а не о том, как именно эти действия выполняются. Наиболее часто используемым действием является перемещение персонажей по полю, поэтому оно должно быть максимально отлажено и не требовать от игрока дополнительных умственных усилий [8].

Основные задачи системы способностей персонажа заключаются в предоставлении разработчику возможности создавать любые действия, которые персонаж может применять на поле, на других юнитов или на самого себя. Такой широкий диапазон опций позволит описать практически любое действие: атаки, заклинания, разговор с другими персонажами и взаимодействие с окружением.

Часть третья. Разработка пользовательского интерфейса

Пользовательский интерфейс (UI) в играх жанра Top-Down RPG выполняет важные функции предоставления игроку информации о состоянии персонажей (здоровье, ресурсы, активные эффекты), обеспечение возможности управления отрядами (выделение, отдача команд), а также отображение контекстной информации об объектах игрового мира (доступные действия, подсказки).

В рамках третьей части работы рассмотрена разработка пользовательского интерфейса для управления персонажами в Top-Down RPG, а конкретно:

- Разработка HUD, отображающего способности выбранного персонажа;
- Интеграция пользовательского интерфейса с разработанными игровыми механиками.

Объектом исследования в данной работе является процесс разработки плагина, расширяющего функционал игрового движка Unreal Engine 5 для возможности создания проектов Top-Down RPG.

Цели работы:

1. Провести обзор коммерчески успешных проектов разных эпох - с точки зрения визуального оформления, а также эволюции систем управления и взаимодействия пользователя с игровым миром.
2. Разработать инструменты разработчика для создания и управления Tilemap - игрового поля, разделённого на клетки.
3. Разработать игрового персонажа с возможностью перемещения по игровому полю, а также разработать систему способностей персонажа;
4. Разработать пользовательский интерфейс: управление камерой и взаимодействие с игровыми персонажами.

1 Исследование существующих проектов

В рамках данной работы было принято решение о разработке Tile-Based Top-Down RPG, то есть игры, игровое поле которой разделено на дискретные клетки (тайлы). Такой подход широко применяется в тактических и пошаговых RPG, поскольку он обеспечивает чёткую структурированность игрового пространства, упрощает реализацию навигации, а также даёт игроку однозначное понимание доступных для перемещения зон и зон действия способностей.

Для обоснования выбранных архитектурных и дизайнерских решений был проведён анализ коммерчески успешных проектов жанра, относящихся к разным эпохам развития видеоигровой индустрии. Анализ выполнялся по двум направлениям:

- **Визуальный анализ** - определение средних размеров клеток игрового поля и персонажей на экране (в пикселях и физических сантиметрах), что позволило выработать рекомендации по масштабированию тайлов для обеспечения комфортного восприятия игроком;
- **Анализ систем управления** - изучение распределения функций между левой и правой кнопками мыши, а также возможности выполнения всех действий через левую кнопку мыши.

1.1 Визуальный анализ

В качестве объектов анализа визуальной составляющей были взяты следующие коммерчески успешные проекты: Warcraft 2(1995); Fallout (1997); Majesty (2000); ХСОМ 2(2016). Были измерены средние размеры клеток поля и размеры персонажей на экране в пикселях, а также в сантиметрах. В качестве инструмента измерения физических метрик был взят монитор с диагональю 24 дюйма и соотношением сторон 16:9, что эквивалентно 51.1 см в ширину и 29.9 см в высоту.

Все полученные измерения представлены в таблице 1, скриншоты рассмотренных проектов представлены на рис.1-4.

Warcraft 2

Год выпуска: 1995

Разрешение: 640x480

В данном проекте была использована квадратная сетка с равными сторонами, а также игровые персонажи и сущности, по размерами сопоставимые с одной игровой клеткой.



Рисунок 1 - Скриншот видеоигры Warcraft 2

Fallout

Год выпуска: 1997

Оригинальное разрешение 640x480

Разрешение при измерении: 1280x720

Хотя все отображаемые здания и клетки «пола» имеют четырехугольную форму, в данном проекте используется шестиугольная сетка.



Рисунок 2 - Скриншоты видеоигры Fallout

Majesty

Год выпуска: 2000

Разрешение: 800x600

В данном проекте игровая сетка не отображается напрямую, но влияет на расстановку внутриигровых зданий и перемещение персонажей по карте. Сетка была измерена с помощью измерения минимального возможного «шага» при выборе позиции нового здания.



Рисунок 3 - Скриншот видеоигры Majesty

X-COM 2

Год выпуска: 2016

Разрешение: 1280x720

В отличие от остальных рассмотренных проектов, в данном используется не ортогональная камера, а камера с перспективой, то есть клетки и персонажи имеют разный размер на экране в зависимости от положения камеры. Для измерения были взяты клетки и персонажи, находящиеся в центре экрана.

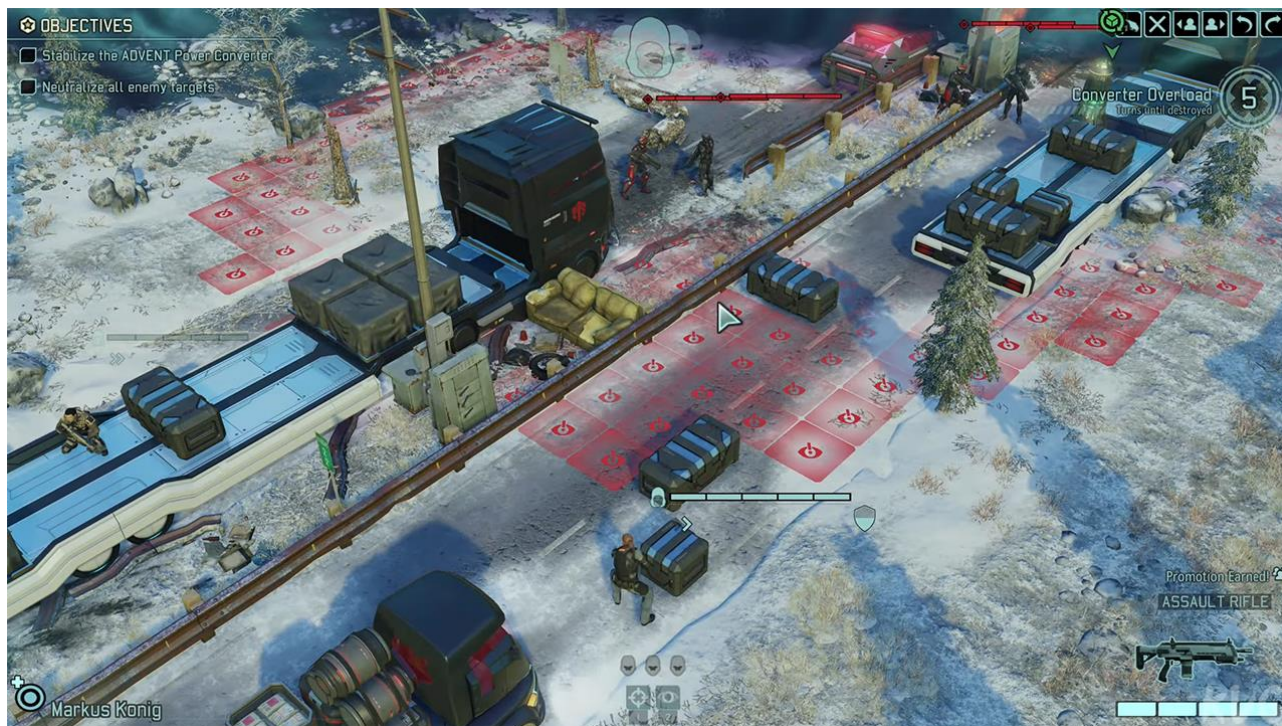


Рисунок 4 - Скриншот видеоигры XCOM 2

Таблица 1

Измерения размеров клеток и персонажей в различных проектах.

Название	Разрешение	Размер сетки, в пикселях	Размер сетки, в см	Размер персонажа, в пикселях	Размер персонажа, в см
Warcraft 2	480x640	34x34	2.11x2.11	34x40	2.11x2.48
Majesty	800x600	34x28	1.69x1.39	30x50	1.49x2.49
Fallout	1280x720	66x35	2.63x1.45	47x88	1.87x3.65
XCOM 2	1280x720	90x64	3,59x2,65	45x90	1.79x3.73

Полученные измерения будут использованы при дальнейшей разработке игрового пространства.

1.2 Анализ пользовательского интерфейса

Для анализа UI были взяты четыре коммерчески успешных проекта, представляющих различные эпохи развития жанра Top-Down RPG: Fallout (1997), Baldur's Gate (1998), XCOM 2 (2016) и Baldur's Gate 3 (2023). Целью анализа являлось изучение эволюции систем управления мышью, а именно - распределения функций между левой и правой кнопками мыши, а также возможности выполнения всех действий через одну кнопку.

Fallout 1 (1997) использует схему управления, построенную на трёх состояниях курсора, которые переключаются правой кнопкой мыши. Левая кнопка мыши (далее ЛКМ) выполняет действие, соответствующее текущему режиму курсора:

- В режиме перемещения ЛКМ выполняет перемещение персонажа в указанную точку;
- В режиме атаки ЛКМ выполняет атаку игрового персонажа по выбранной цели.
- В режиме взаимодействия ЛКМ выполняет различные действия взаимодействия с игровым миром: разговор с неигровым персонажем, открытие дверей, использование предметов.

Особый интерес представляет режим взаимодействия. При удержании левой кнопки мыши в этом режиме открывается круговое меню с дополнительными иконками действий: «говорить», «осмотреть», «использовать», «взять» и другими. Это позволяет игроку выбирать специфическое действие с неигровым персонажем или объектом, при том, что обычный клик левой кнопкой мыши в режиме взаимодействия запускает стандартное действие (например, разговор).

В Baldur's Gate (1998) прослеживается чёткое разделение функций между левой и правой кнопками мыши. Игрок наводит курсор на объект и интерфейс динамически отображает, какое действие будет выполнено при клике ЛКМ посредством изменения иконки указателя мыши. Клик ПКМ, в свою очередь, открывает детальное контекстное меню с полным списком доступных действий для выбранного объекта или персонажа.

В XCOM 2 (2016) левая кнопка мыши аналогично используется для выбора персонажа, выбора цели и подтверждения действий. Правая кнопка мыши выполняет функцию отмены выбора и открытия дополнительной информации. В данном проекте также присутствует оригинальная механика, использующая обе кнопки мыши: при выборе определенного персонажа - солдата ближнего боя - игроку даётся возможность совместить перемещение с атакой за счёт одновременного нажатия обоих кнопок мыши по цели.

Baldur's Gate 3 (2023), как самый современный представитель жанра Top-Down RPG представляет собой самый релевантный объект для анализа, так как он разрабатывался с учётом современных стандартов UI и UX. Используется уже устоявшийся принцип: нажатие левой кнопки мыши для основного действия, зависящего от контекста (перемещение, разговор с персонажем, взаимодействие с объектом) и нажатие правой кнопки мыши для вызова контекстного меню со списком всех доступных действий.

Проведённый анализ четырёх коммерчески успешных проектов разных эпох выявляет чёткую эволюционную линию в развитии систем управления Top-Down RPG. Ранние проекты требовали ручного переключения режимов курсора, что создавало дополнительную когнитивную нагрузку на игрока. В поздних проектах устоялась модель контекстно-зависимого взаимодействия.

На основе анализа принято решение реализовать в разрабатываемой системе взаимодействия модель, аналогичную XCOM 2: левая кнопка мыши будет выполнять контекстно-зависимое основное действие (выбор персонажа, выбор точки для навигации, выбор и использование действия персонажа), а правая кнопка мыши будет выполнять функцию отмены выбора или действия.

2 Обзор используемых технологий

Расширение инструментария Unreal Engine 5 для создания проектов Top-Down RPG требует глубокого понимания и использования различных технологий, поставляемых с игровым движком. Ниже представлен обзор технологий, использованных в проекте.

Для разработки всех программных компонентов плагина использован язык C++ в его реализации, предоставляемой Unreal Engine 5. Ключевым отличием C++ в Unreal Engine является наличие системы рефлексии, реализованной через специальные макросы и автоматически генерируемый код. Системы сборки проекта *Unreal Build Tool* (UBT) и *Unreal Header Tool* (UHT) обрабатывают заголовочные файлы перед компиляцией, распознавая макросы и создавая дополнительные файлы, содержащие служебную информацию. Макросы *UCLASS*, *USTRUCT*, *UFUNCTION* и др. позволяют расширять разрабатываемые объекты с помощью языка визуального скриптинга Unreal Engine *Blueprints*.

Для внедрения создаваемых инструментов в редактор движка использованы и расширены следующие классы [9]:

UInteractiveTool - класс, описывающий поведение инструментов внутри Editor Mode. Инструменты отвечают за конкретные действия пользователя: рисование клеток, удаление, заливка, добавление объектов в сцену и т.д.

FEditorModeTools - класс-менеджер, отслеживающий зарегистрированные в редакторе режимы, а также координирует обработку ввода между режимами.

Созданный плагин разделён на два независимых модуля: *TilemapToolRuntime* и *TilemapToolEditor*.

Модуль *TilemapToolRuntime* содержит код, необходимый для работы игры в финальной сборке. В него входят все ключевые игровые сущности: базовый класс сетки (*ATilemapBase*), интерфейс для взаимодействия с объектами на тайлах (*ITileContentInterface*), а также классы персонажа *ATileContentCharacter* и система его способностей. Благодаря тому, что модуль промаркирован в файле описания плагина (.uplugin) типом *Runtime*, все его компоненты будут доступны как в редакторе во время разработки, так и в готовом приложении [10].

Модуль *TilemapToolEditor* содержит исключительно код инструментов, предназначенных для работы в редакторе Unreal Engine. В него вынесены все классы расширения редактора: пользовательский режим *UTileMapToolEditorMode*, интерактивные инструменты *UTileMapToolEditCellsTool*, *UTileMapToolPlaceActorsTool* и *UTileMapToolDeleteActorsTool*. Данный модуль помечен типом *Editor*, что гарантирует: его код не попадает в финальную сборку, предназначенную для игроков. Согласно правилам лицензионного соглашения, код редактора строго исключён из поставляемых приложений [11].

Для перемещения персонажей по игровому миру в Unreal Engine используется система, основанная на генерации навигационных сеток. *NavMesh* - представление доступного для перемещения пространства в виде выпуклых многоугольников. Подсистема *Navigation System* автоматически генерирует *NavMesh* на основе геометрии уровня и специальных областей навигации [12].

Для создания пользовательских интерфейсов в Unreal Engine используется система *UMG* (Unreal Motion Graphics). *UMG* позволяет разрабатывать интерфейсы как в визуальном редакторе (Widget Blueprint), так и с помощью C++ [13].

3 Разработка игрового поля

На языке C++ был разработан класс ATilemapBase, наследующий от класса AActor, предоставляемого движком Unreal Engine. Этот класс включает в себя различные параметры сетки, не зависящие от формы ячеек, а также описывает интерфейс взаимодействия для других классов: управление содержимым ячеек, получение данных о навигации, а также управление визуальной составляющей сетки: окрашивание и подсвечивание ячеек.

Листинг 1

Декларация класса ATilemapBase.

```
0  UCLASS()
1  class TILEMAPTOOL_API ATilemapBase : public AActor
2  {
3      GENERATED_BODY()
4  public:
5      ATilemapBase();
6      virtual FVector2D GetTileIndexByWorldCoordinates(FVector Coordinates);
7      virtual ITileContentInterface* GetTileContentByIndex(FVector2D Index);
8      ITileContentInterface* GetTileContentByIndex(int X, int Y);
9      ETileStatus GetTileStatusByIndex(int X, int Y);
10     FVector GetTilePositionByIndex(int X, int Y)
11     bool SetTileContentByIndex(int X, int Y, ITileContentInterface* Content);
12     bool SetTileStatusByIndex(int X, int Y, ETileStatus Status);
13     bool SetTilePositionByIndex(int X, int Y, FVector Position);
14     virtual bool bTileExistsByWorldCoordinates(FVector Coordinates);
15     virtual TArray<FVector2D> Navigate(FVector2D StartIndex, FVector2D EndIndex);
16
17     virtual void OnConstruction(const FTransform& Transform) override;
18
19     virtual void SelectTile(int X, int Y){};
20     virtual void paintTile(int X, int Y, FTileVisualProperties Properties);
```

```

21     virtual void paintAllTiles(FTileVisualProperties Properties);
22     virtual void resetTilePaint();
23     virtual void paintRange(FVector2D CenterIndex,
int32 range);
24
25     virtual void ExplicitSerialize();
26     virtual void ExplicitDeserialize();
27
28     FVector2D getDimensions(){return FVector2D(DimensionX, DimensionY);}
29 protected:
30     TArray<TArray<ITileContentInterface*>> TileContentMatrix;
31     TArray<TArray<ETileStatus>> TileStatusMatrix;
32     TArray<TArray<FVector>> TilePositionMatrix;
33     UInstancedStaticMeshComponent* ISMCTileMesh;
34     URuntimeVirtualTexture* TilemapGridRVT;
35 };

```

Для возможности определения положения клеток на игровом поле было решено создавать «каркас» сетки внутри сцены. Геометрия каркаса не будет видна игроку напрямую, но «тень» сетки будет проецироваться на игровую поверхность. На рис.5 представлены каркасы сеток.

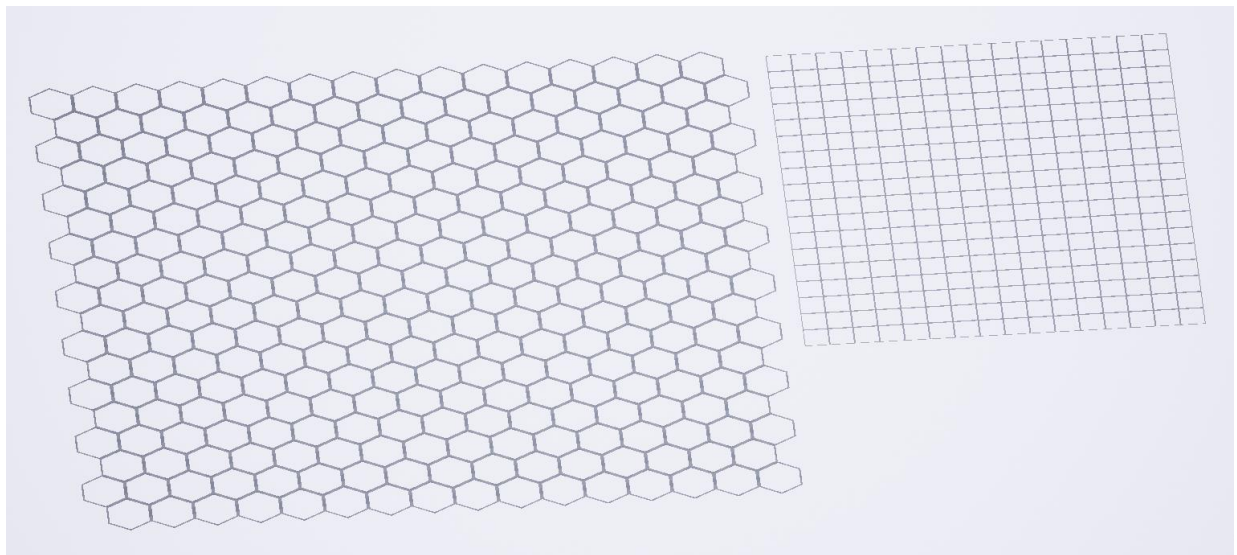


Рисунок 5 - Сетки, состоящие из отдельных 3д моделей клеток

Для наложения сетки на неровные поверхности (холмы, мосты, овраги) используется технология Runtime Virtual Texture (RVT) [14]. Материал ландшафта берет данные с виртуальной текстуры, в которую каркас сетки записывает изображение клеток. Это обеспечивает визуальную интеграцию сетки с окружением без ручного выравнивания. Проекция сетки продемонстрирована на рис.6.

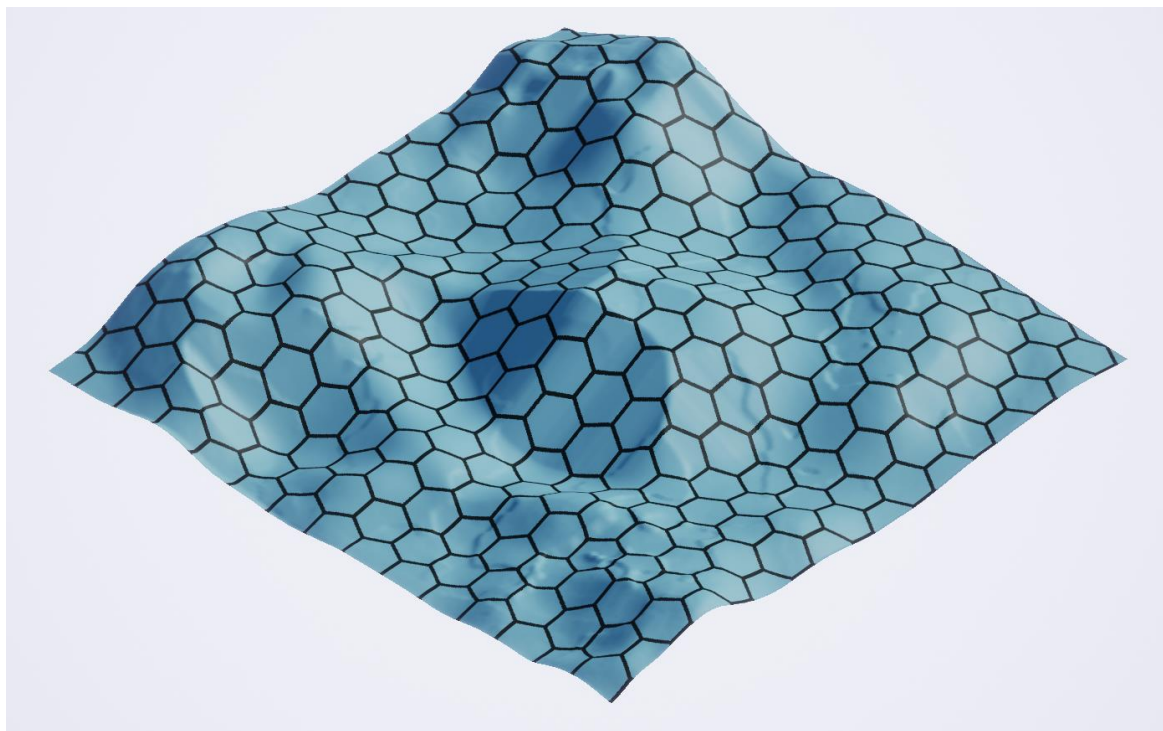


Рисунок 6 - Проекция разработанной сетки на ландшафт произвольной формы

Использование представления сетки в виде геометрии так же позволяет решить вопрос определения нужной ячейки в разных ситуациях игрового процесса. При вводе игроком команды, которая требует выбора ячейки проводятся две трассировки лучом. Первая трассировка позволяет получить точку на неровной игровой поверхности, на которую спроецирована сетка. Вторая трассировка происходит из найденной точки горизонтально в сторону каркаса сетки. Класс `UninstantedStaticMeshComponent` даёт возможность проводить трассировку по сразу по всему массиву инстансов геометрии, что позволяет второй трассировке получить координаты искомой ячейки [15]. На рис.7 представлена схема нахождения координат ячейки по команде пользователя.

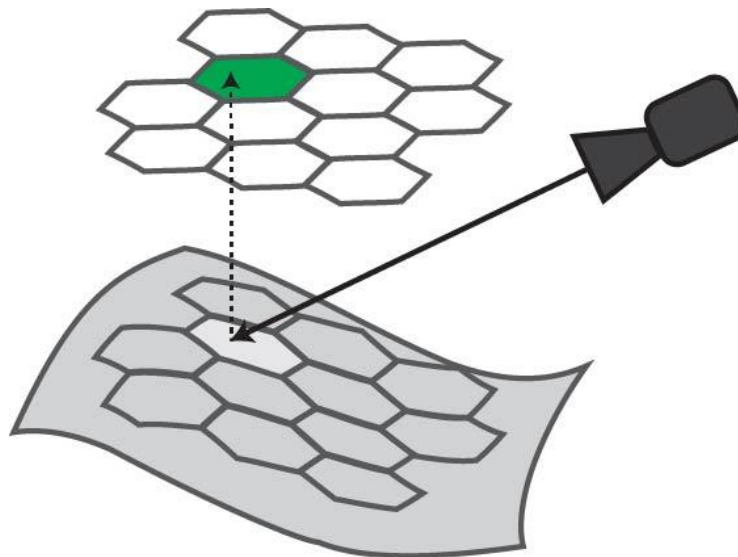


Рисунок 7 - Определение координат ячейки сетки, проецируемой на игровую поверхность

3.1 Хранение в памяти

Для сохранения результатов манипуляций, производимых с сеткой внутри редактора Unreal Engine, была реализована сериализация класса ATilemapBase. В бинарный файл, задаваемый пользователем, записываются параметры сетки и ячеек, а также уникальные идентификаторы актёров, прикрепленных к сетке. При десериализации происходит поиск по идентификатором актёров, уже присутствующих в сцене. Найденные актёры привязываются к сетке, а отсутствующие создаются заново по сохранённым параметрам. При запуске игры сетки используют сохраненный файл для корректного создания игрового поля. Для удобства работы функции ExplicitSerialize и ExplicitDeserialize были вынесены в визуальный интерфейс редактора с помощью макроса UFUNCTION. В листинге 2 представлена функция для сериализации сетки, использующая классы Unreal Engine. На рис.8 изображены полученные функции, вынесенные в виде кнопок в интерфейс редактора.

Листинг 2 - Функция сериализации сетки.

```

0 void ATilemapSquare::ExplicitSerialize()
1 {
2     UpdateFullFilePath();
3     SavedTileContents.Empty();
4     for (int Index1=0; Index1<DimensionX; Index1++)
5         for (int Index2=0; Index2<DimensionY; Index2++)
6         {

```

```

7      AActor* ContentActor = Cast<AActor>(TileContentMa-
trix[Index1][Index2]);
8      if (IsValid(ContentActor))
9      {
10         FTileContentSaveData Data;
11         Data.ActorGuid = TileContentMatrix[Index1][In-
dex2]->GetGuid();
12         Data.ActorClass = ContentActor->GetClass();
13         Data.SpawnTransform = ContentActor->GetTrans-
form();
14         Data.Index=FVector2D(Index1,Index2);
15         SavedTileContents.Add(Data);
16     }
17 }
18 TArray<uint8> Buffer;
19 FMemoryWriter Writer(Buffer);
20 FObjectAndNameAsStringProxyArchive ProxyWriter(Writer,
false);
21 ProxyWriter << DimensionX;
22 ProxyWriter << DimensionY;
23 ProxyWriter << TilePositionMatrix;
24 ProxyWriter << TileStatusMatrix;
25 ProxyWriter << SavedTileContents;
26 FileHelper::SaveArrayToFile(Buffer, *FullFilePath);
27 }

```

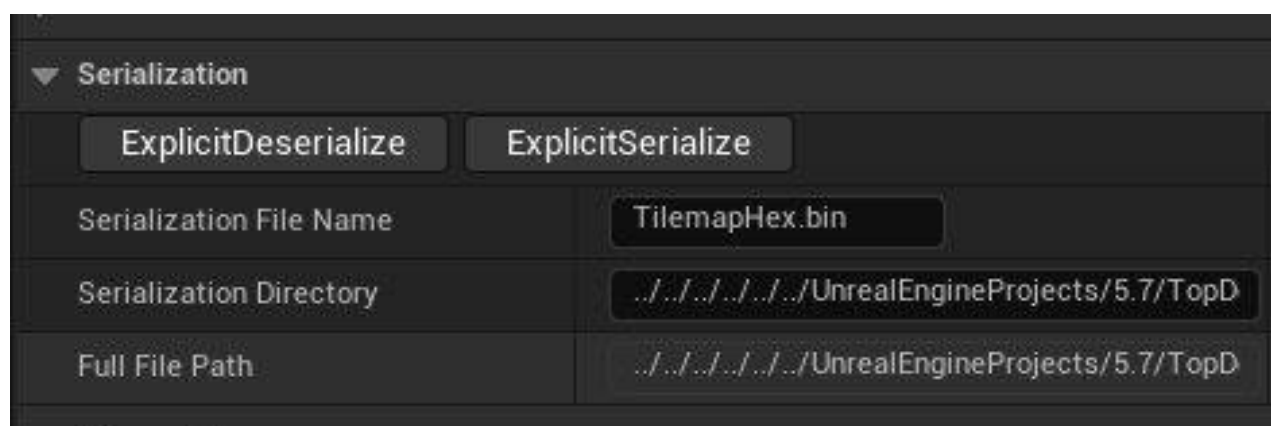


Рисунок 8 - Функции для сериализации и десериализации сетки, вынесенные в графический интерфейс редактора

3.2 Навигация

Разработанные функции `ATilemapBase::Navigate` и `ATilemapBase::Heuristic` возвращают путь между двумя ячейками и оценку расстояния между ячейками соответственно. Они могут быть переопределены в наследующих классах для поддержки произвольных форм сетки, а также альтернативных алгоритмов навигации. В данном случае для навигации по сетке выбран алгоритм A^* и реализована навигация по квадратной и шестиугольной сетке, реализации отличаются функцией для оценки расстояния между клетками и способом определения соседних клеток внутри алгоритма A^* [16]. На рис.9 продемонстрировано построение пути между двумя ячейками сетки.

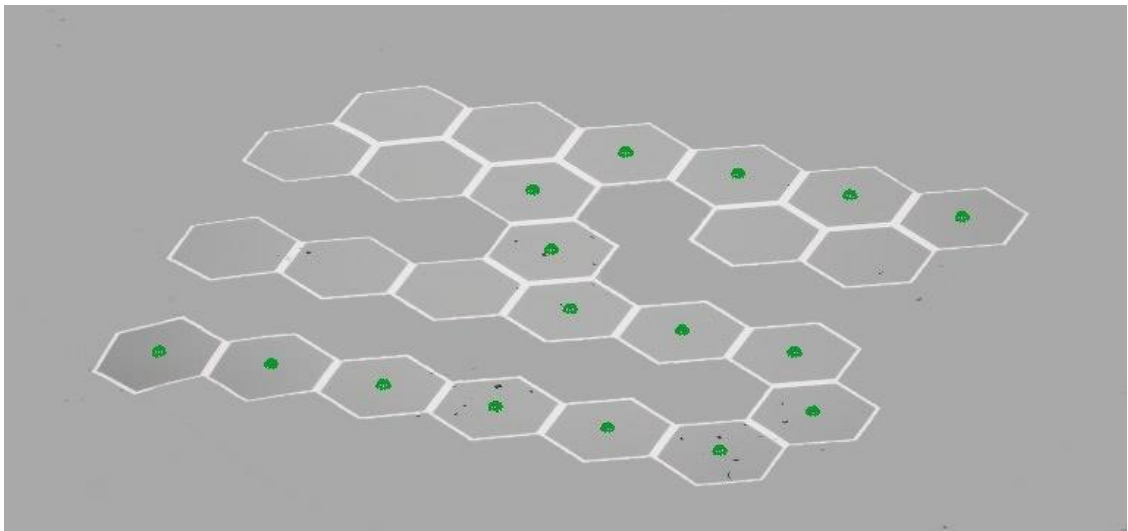


Рисунок 9 - Построение пути между крайними ячейками сетки

3.3 Оптимизация отображения сетки

При разработке тайловых игровых полей размером до нескольких тысяч клеток (например поле 100 на 100) вопрос производительности отрисовки становится критическим. Каждая клетка сетки представляет собой самостоятельную геометрическую единицу, которая должна быть отображена в мире.

В разработанном плагине выбрана следующая технология оптимизации: использование `UInstancedStaticMeshComponent` [17]. ISMC позволяет хранить тысячи трансформаций (структура с описанием положения, поворота и масштаба) внутри одного компонента и отрисовывать все инстансы одной геомет-

рии за один draw call. Для сетки, имеющей единую форму ячеек достаточно одного экземпляра ISMC, в который добавляются трансформации для каждой существующей клетки. Основные преимущества ISMC, реализованные в проекте:

- Снижение количества draw calls: все клетки сетки рендерятся за один вызов независимо от их количества.
- Экономия памяти: позиции, повороты и масштабы хранятся в компактном буфере, а не в виде отдельных компонентов.
- Встроенная трассировка: `UInstancedStaticMeshComponent` предоставляет методы для выполнения трассировки лучей по всем инстансам и получения индекса поражённого экземпляра.

4 Разработка режимов редактора

Редактор Unreal Engine предоставляет разработчикам несколько готовых режимов: размещение объектов, создание ландшафтов, моделирование 3д геометрии и другие. Было решено создать новый режим редактора, объединяющий в себе различные функции (Tools) для управления сеткой. Создан класс `UTileMapToolEditorMode`, наследующий от класса `UEdMode`, нужного для определения новых режимов редактора, а также классы, описывающие функционал новых команд редактора [18]:

- `UTileMapToolEditCellsTool` - изменение статуса ячеек.
- `UTileMapToolPlaceActorsTool` - привязка объектов к сетке.
- `UTileMapToolDeleteActorsTool` - удаление объектов, привязанных к сетке.

4.1 Настройка статусов ячеек

В зависимости от действий игрока, логики игры или этапа построения уровня клетки сетки могут приобретать различные статусы, влияющие на навигацию, возможность размещения объектов и визуальное представление. Выбран следующий набор статусов, описывающий базовый минимум различных состояний нужных для функционирования навигации и размещения объектов на сетке:

- Пустая ячейка;
- Ячейка, занятая актёром, это может быть предмет, здание или персонаж;
- Отключенная ячейка, это состояние полезно для удаления клеток на этапе создания игровой карты, например для областей, где не предполагается нахождение объектов или персонажей.

На основе класса `UInteractiveTool`, описывающего базовое поведение инструментов редактора, создан класс `UTileMapToolEditCellsTool`. Он описывает следующее поведение: при нажатии пользователя по клетке происходит определение координат интересующей клетки, и через публичный интерфейс найденной сетки изменяется значение матрицы статусов клеток. На рис.10 представлен графический интерфейс данного инструмента, а также клетки поля в различных состояниях, показанных различными цветами.

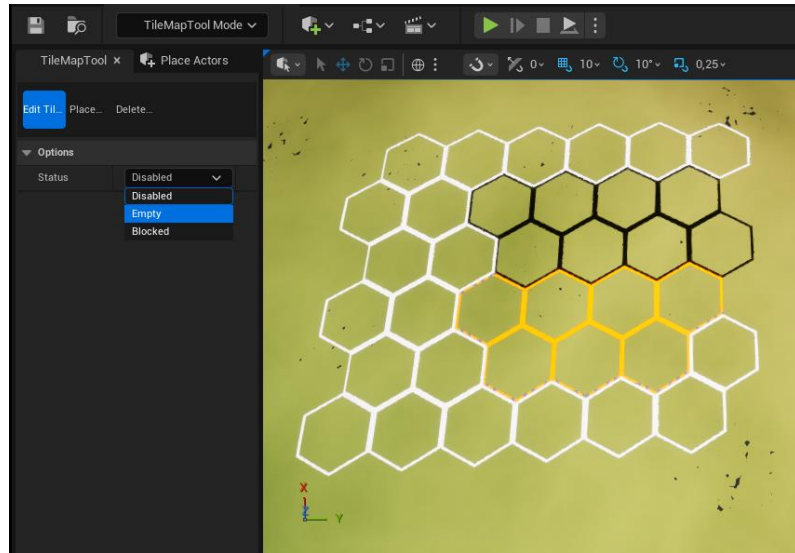


Рисунок 10 - Интерфейс редактора клеток (слева) и отображение клеток различных статусов.

4.2 Размещение объектов на сетке

Для организации взаимодействия с объектами, которые могут находиться на сетке разработан класс `ITileContentInterface`, описывающий интерфейс взаимодействия с сеткой. Определение класса представлено в листинге 3. С помощью макроса `UINTERFACE` классы созданные в Unreal Engine Blueprints могут наследовать описанный интерфейс, что значительно ускоряет создание вариаций игровых объектов во время разработки. Использование Blueprints также позволяет выбирать объект для размещения напрямую из браузера ассетов внутри редактора Unreal Engine [19].

Листинг 3 - Определение интерфейса `UTileContentInterface`.

```
0  UINTERFACE(Blueprintable)
1  class UTileContentInterface : public UInterface
2  {
3      GENERATED_BODY()
4  };
5
6  class ITileContentInterface
7  {
8      GENERATED_BODY()
9  public:
10     virtual FGuid GetGuid() const = 0;
11     virtual void SetGuid(const FGuid& NewGuid) = 0;
12     virtual bool PlaceOnTilemap(ATilemapBase* Tilemap, FVector2D Index) = 0;
13     virtual void RemoveFromTilemap() = 0;
```

```

14     virtual ATilemapBase* GetTileMap() = 0;
15     virtual FVector2D GetTilemapIndex() = 0;
16     virtual AActor* GetActor()=0;
17 };

```

На основе класса UInteractiveTool разработан класс UTileMapToolPlaceActorsTool. Он описывает следующий функционал: при нажатии пользователя по клетке происходит поиск подходящего объекта для размещения в браузере ассетов; если подходящий ассет найден происходит трассировка интересующей клетки; если клетка найдена и имеет статус «пустая» объект создается и прикрепляется к сетке с помощью функций, описанных в интерфейсе ITileContentInterface.

Для большего художественного контроля над размещаемыми объектами в интерфейс инструмента добавлены следующие параметры:

- Разброс угла поворота;
- Разброс масштаба;
- Сдвиг позиции в пределах клетки.

Эта опция особенно полезна при массовом размещении однотипных предметов на сетке, таких как деревья в лесу, камни на равнине, кусты или мусор на городской улице. Без случайных вариаций объекты могут иметь ненатуральный эффект, сильно ломающий реалистичность изображения.

На рис.11 представлен графический интерфейс инструмента для размещения объектов.

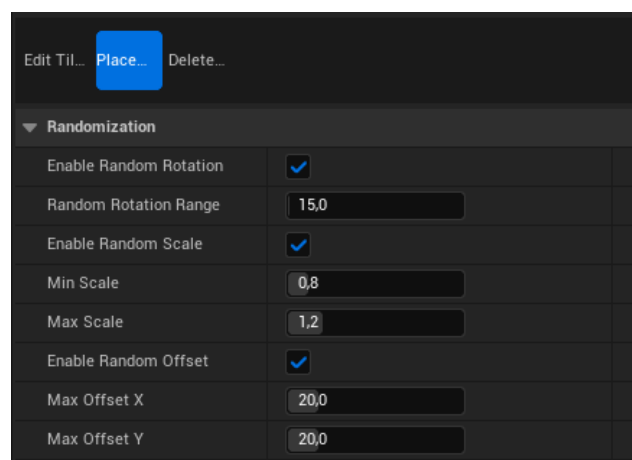


Рисунок 11 - Графический интерфейс инструмента для размещения объектов на сетке.

На рисунках 12 и 13 изображена сетка с различными объектами, размещенными «ровно» и со случайными отклонениями соответственно. Можно заметить, что случайные вариации размещения успешно скрывают факт повторения геометрии.

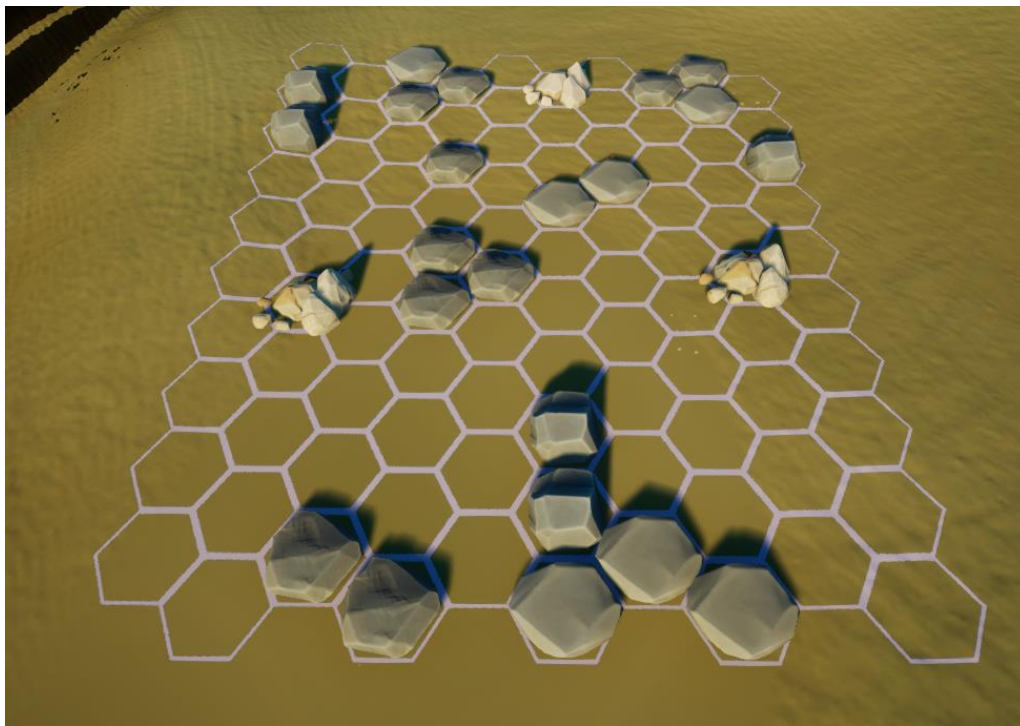


Рисунок 12 - Различные объекты, размещенные на сетке «ровно»

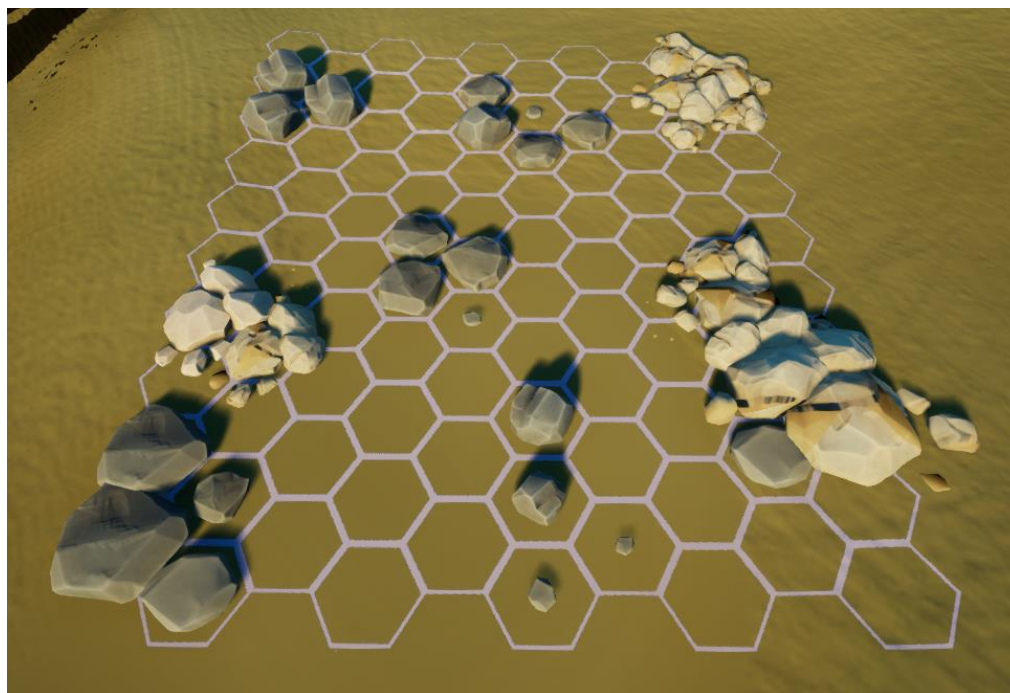


Рисунок 13 - Различные объекты, размещенные на сетке со случайными вариациями в позиции

Для удаления ранее размещённых объектов разработан класс `UTileMapToolDeleteActorsTool`, наследующий от `UInteractiveTool`. При клике по клетке, содержащей актёр, реализующий интерфейс `ITileContentInterface`, инструмент выполняет следующие действия:

- Вызов метода `RemoveFromTilemap()` у интерфейса.
- Обновление матрицы статусов `TileStatusMatrix` на значение `Empty`.
- Обнуление соответствующей ячейки в матрице содержания `TileContentMatrix`.

5 Разработка игрового персонажа

С использованием разработанного интерфейса `ITileContentInterface` был создан класс `ATileContentCharacter`, наследующий от `ACharacter`. `ACharacter` - стандартный класс Unreal Engine, который предоставляет готовую функциональность для управляемых персонажей: скелетную анимацию, систему навигации и перемещения, а также настройки передвижения (скорость ходьбы, ускорение, скорость поворота и т.д.) [20]. Использование этого класса в качестве основы позволило не реализовывать базовую логику движения с нуля.

Перемещение игрового персонажа по сетке происходит следующим образом: игрок выбирает клетку, в которую необходимо переместиться, и функция `Navigate` соответствующей сетки возвращает последовательность координат ячеек, формирующих маршрут. На основе полученных координат определяются мировые координаты (точки на ландшафте), через которые должен пройти персонаж. Затем контроллер персонажа получает эти точки и последовательно перемещает персонажа по указанному пути с помощью технологий `NavMesh`, предоставляемых Unreal Engine. На рис. 14 схематично представлен процесс навигации персонажа.

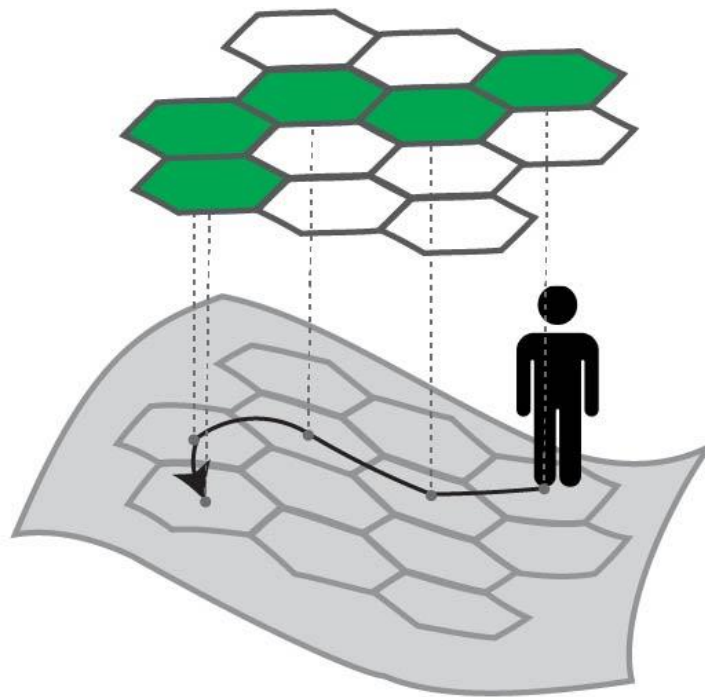


Рисунок 14 - Навигация персонажа по неровной поверхности с помощью NavMesh

5.1 Разработка системы действий персонажа

В ролевых играх функционал персонажа зачастую не ограничивается перемещением по полю. Помимо навигации могут быть боевые способности (атаки, заклинания), взаимодействие с интерактивными объектами (двери, сундуки, ловушки) а также взаимодействие с другими персонажами (например, в виде диалоговой системы). Действия могут адресоваться не только к конкретным целям (врагам или предметам), но и к пустым сеткам игрового поля - например для установки предметов на игровое поле. В связи с этим была разработана система действий персонажа, позволяющая игроку выбирать действие и применять его к клетке в выбранном радиусе вокруг персонажа. Такая система покрывает большое число типовых сценариев игрового процесса в жанре, а также открыта для расширения.

Для управления способностями персонажа был разработан компонент *UCharacterAbilitySystem*, наследующий от стандартного *UActorComponent*. *UCharacterAbilitySystem* хранит в себе набор доступных способностей и при расширении может обрабатывать логику, связанную с выбором способностей, например “Cooldown”, то есть блокировку способности на какое-то время непосредственно после применения [21].

Для представления отдельной способности персонажа был создан класс *UCharacterAbility*, унаследованный от *UObject*. Класс содержит следующие ключевые свойства способности:

- Range - целочисленное поле, определяющее максимальную дальность применения способности в клетках;
- Name - строковое поле, хранящее название способности для отображения в графическом интерфейсе;
- Icon - текстурное поле (тип *UTexture2D*), содержащее иконку способности, которая будет отображаться на панели выбора действий.

Основным методом класса является *Execute*, принимающий ссылку на тайловую сетку и координаты клетки, к которой применяется способность. Метод

реализован как *BlueprintNativeEvent*, что позволяет переопределить его логику в визуальных скриптах *Blueprints* [22].

Полученный компонент *UCharacterAbilitySystem* может быть добавлен к любому актёру через стандартный интерфейс редактора Unreal Engine. Благодаря спецификаторам *BlueprintSpawnableComponent* и *EditInlineNew*, компонент отображается в списке доступных компонентов и позволяет настраивать список способностей прямо в редакторе, без написания дополнительного кода. Разработчик может добавлять, удалять и изменять параметры способностей для конкретно выбранного персонажа сцены. На рис.15 показан пример настройки компонента *UCharacterAbilitySystem* в редакторе Unreal Engine [23][24].

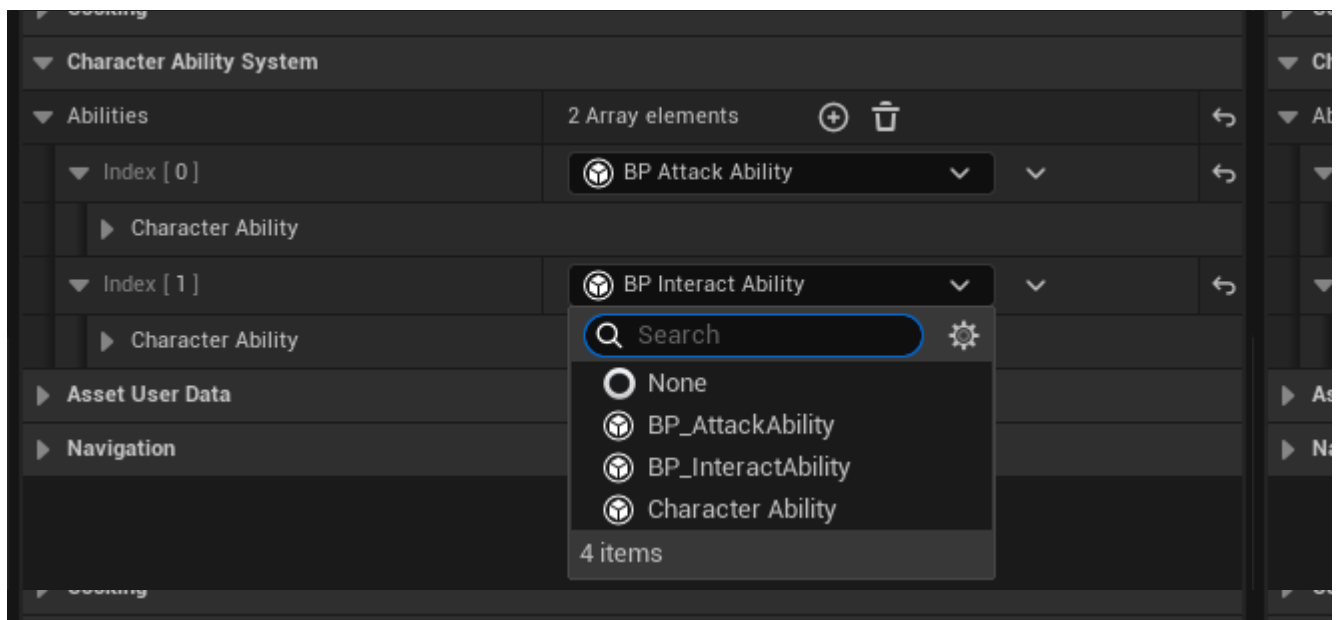


Рисунок 15 – Настройка системы способностей внутри редактора

6 Разработка пользовательского интерфейса

Пользовательский интерфейс в данном проекте представлен двумя сущностями. Первая - камера с видом сверху, позволяющая перемещаться по игровому полю, приближать и отдалять вид, а также менять угол обзора. Вторая - HUD, через который игрок получает информацию о возможных действиях персонажей и подтверждает их применение.

6.1 Управление камерой

Для настройки управления камерой был расширен класс *APawn*, отвечающий за игровых персонажей, которые могут быть контролируемы игроком. В класс-наследник были добавлены компоненты *SpringArm* (рычаг, на который крепится камера) и *Camera*. В настройках компонента *Camera* был выбран режим *Orthographic*, то есть режим отрисовки без перспективы [25]. На рис.16 показан вид сверху, который даёт разработанная сущность.

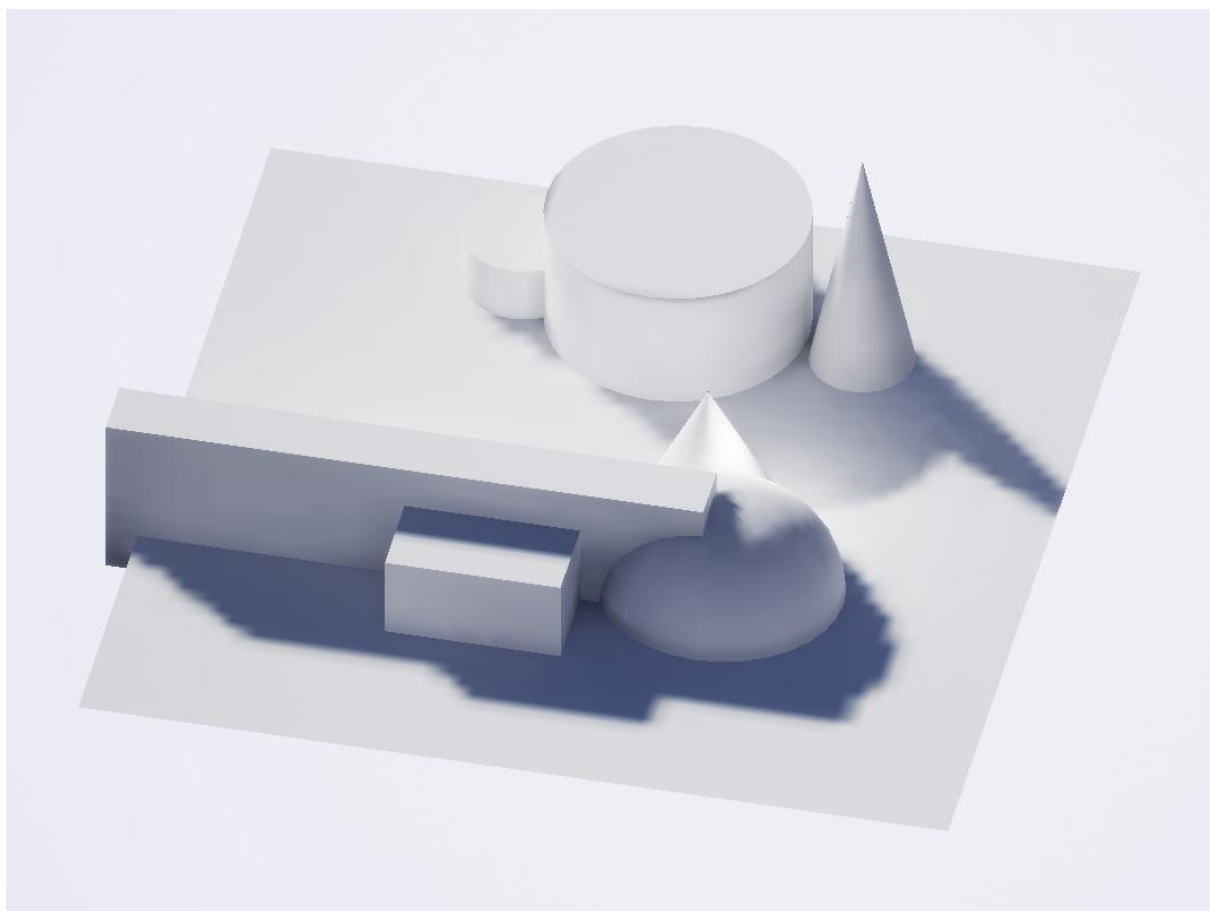


Рисунок 16 - Набор геометрических фигур отображенный в ортографической проекции с помощью разработанной камеры

На визуальном языке программирования *Blueprints* была реализована логика перемещения камеры в горизонтальной плоскости и также поворота вокруг своей оси в зависимости от ввода пользователя, она представлена на рис.17. Решение реализовывать управление камерой посредством *Blueprints* обосновано тем, что тонкости параметров контроля могут проходить множество итераций в процессе гейм-дизайна для достижения нужного «ощущения» (Game Feel). Возможность изменять логику управления внутри редактора Unreal Engine убирает время, нужное на компиляцию проекта и значительно ускоряет итеративность [26].

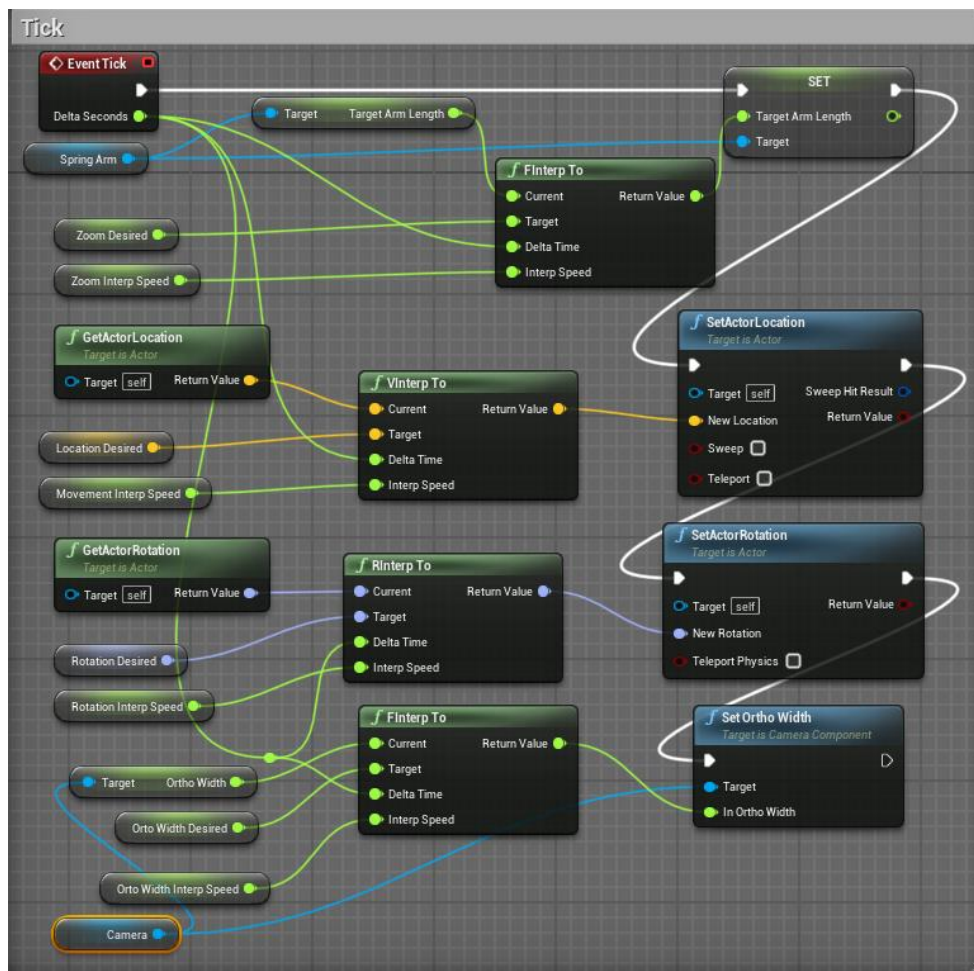


Рисунок 17. *Blueprints* - код, отвечающий за плавное перемещение камеры.

Одной из распространённых проблем, связанных с управлением камерой в играх с видом сверху, является ситуация, когда камера, приближаясь к игровому персонажу или следуя за ним, оказывается слишком близко к вертикальным препятствиям - стенам зданий, скальным выступам, колоннам и другим объектам

окружения. В таких случаях препятствие полностью перекрывает обзор, заслоня важные элементы игрового поля, что может дезориентировать игрока. Для решения этой проблемы был разработан шейдер, который делает стены частично прозрачными в момент, когда камера подходит к ним на критически близкое расстояние.

Вместо обычного альфа-смешивания (alpha blending), в шейдере используется техника dithering (дизеринг). По мере приближения камеры к препятствию значение пиксельной прозрачности преобразуется в частоту отбрасывания пикселей через механизм маски (masked material). Глаз игрока воспринимает множество видимых и невидимых пикселей как полупрозрачность, при этом значительно повышается производительность относительно обычного полупрозрачного шейдера.

Шейдер, реализованный через визуальный язык программирования Material Editor, представлен на рис. 23. Работа шейдера представлена на рис. 24.

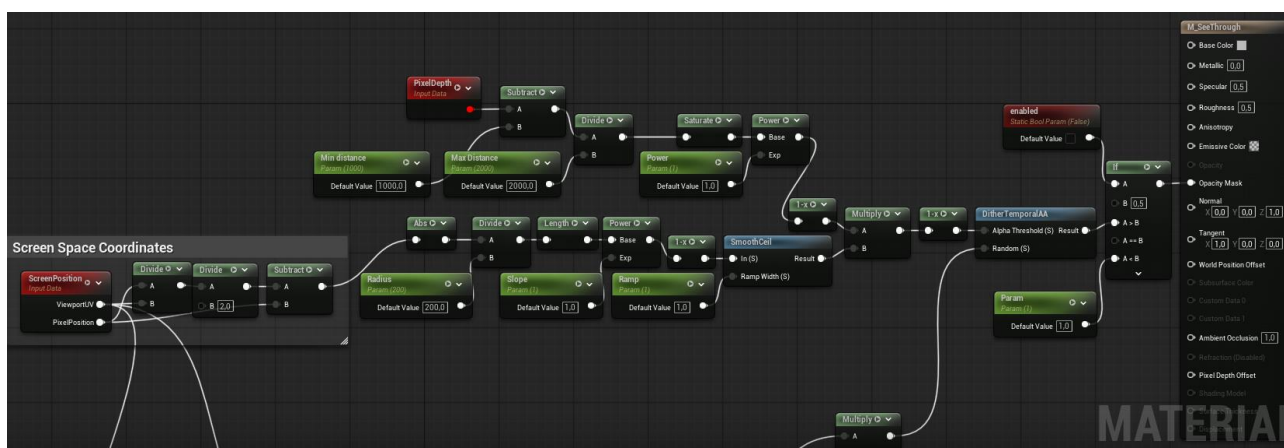


Рисунок 23 – Код шейдера, отвечающий за создание маски

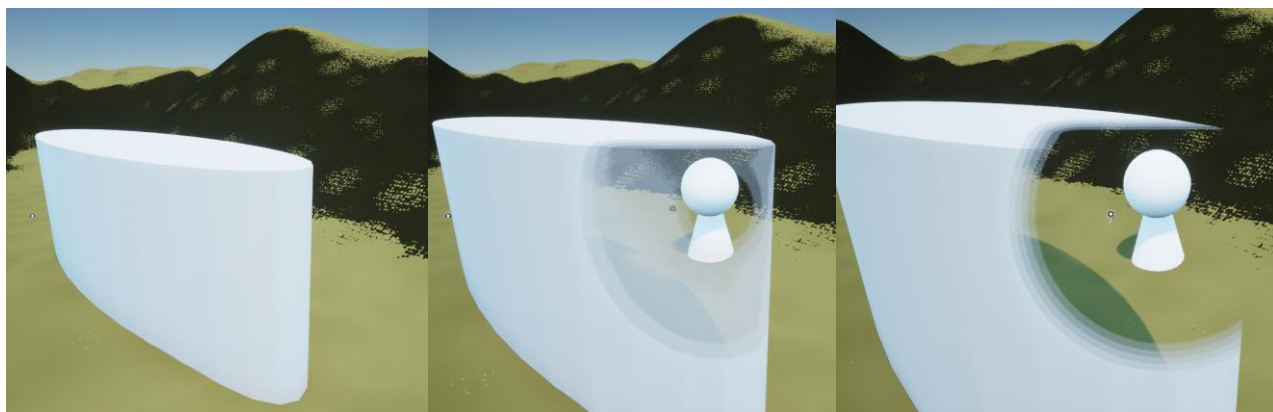


Рисунок 24 – Препятствие становится прозрачным по мере приближения к камере

6.2 Система состояний игрока

В процессе игры в Top-Down RPG игрок последовательно выполняет различные действия: выбирает персонажа, указывает цель для атаки, отдаёт команду на перемещение или использование способности. Каждое из этих действий требует от системы разной интерпретации одного и того же ввода - клика левой кнопкой мыши. Например, в нейтральном состоянии клик по персонажу должен привести к его выбору, а клик по пустой клетке игрового поля - к перемещению выбранного персонажа. Однако, если у игрока уже выбран персонаж и активирован режим выбора цели для способности, тот же самый клик по клетке должен интерпретироваться как применение способности, а не как перемещение.

Для корректной обработки ввода в зависимости от текущего игрового контекста была разработана система состояний пользователя, реализованная в классе `TopDownRPGCharacterController`, наследующем от `APlayerController`. `APlayerController` - стандартный класс Unreal Engine, предназначенный для связи между действиями игрока (ввод с клавиатуры, мыши, геймпада) и поведением управляемого персонажа или других объектов игрового мира. Разработанная система описывает следующие состояния:

`None` - ничего не выбрано (нейтральное состояние). В этом состоянии система ожидает первого действия игрока: выбора персонажа или взаимодействия с объектом. Любой клик по игровому полю, не попадающий в интерактивные объекты, игнорируется, либо отображает подсказку.

`UnitSelected` - игровой персонаж выбран (ожидание команды). Система помнит, какой именно юнит находится под контролем игрока. В этом состоянии клик по пустой клетке вызывает перемещение выбранного персонажа, клик по врагу - атаку (или переход в режим выбора способности, если атака требует подтверждения), клик по интерактивному объекту - взаимодействие с ним. Также в этом состоянии на экране отображается панель способностей выбранного персонажа.

`ActionTargeting` - выбрана способность персонажа (ожидание выбора цели). Игрок активировал одну из способностей, система ожидает, на какую

клетку или противника она будет применена. В этом состоянии клик по допустимой цели активирует способность, клик по недопустимой цели игнорируется или сопровождается звуковым сигналом, клик правой кнопкой мыши отменяет выбор способности и возвращает систему в состояние UnitSelected.

Busy - выбранный персонаж выполняет действие (ожидание завершения действия). В этом состоянии персонаж занят: он перемещается по маршруту, проигрывает анимацию атаки или заклинания, либо находится под действием длительной способности. Все клики мыши в этом состоянии игнорируются или ставятся в очередь для выполнения после завершения текущего действия. Правая кнопка мыши может прерывать действие (если это разрешено типом способности).

На рис.18 представлена схема переходов между различными состояниями игрока с указанием событий, вызывающих переход (клик ЛКМ, клик ПКМ, завершение действия).

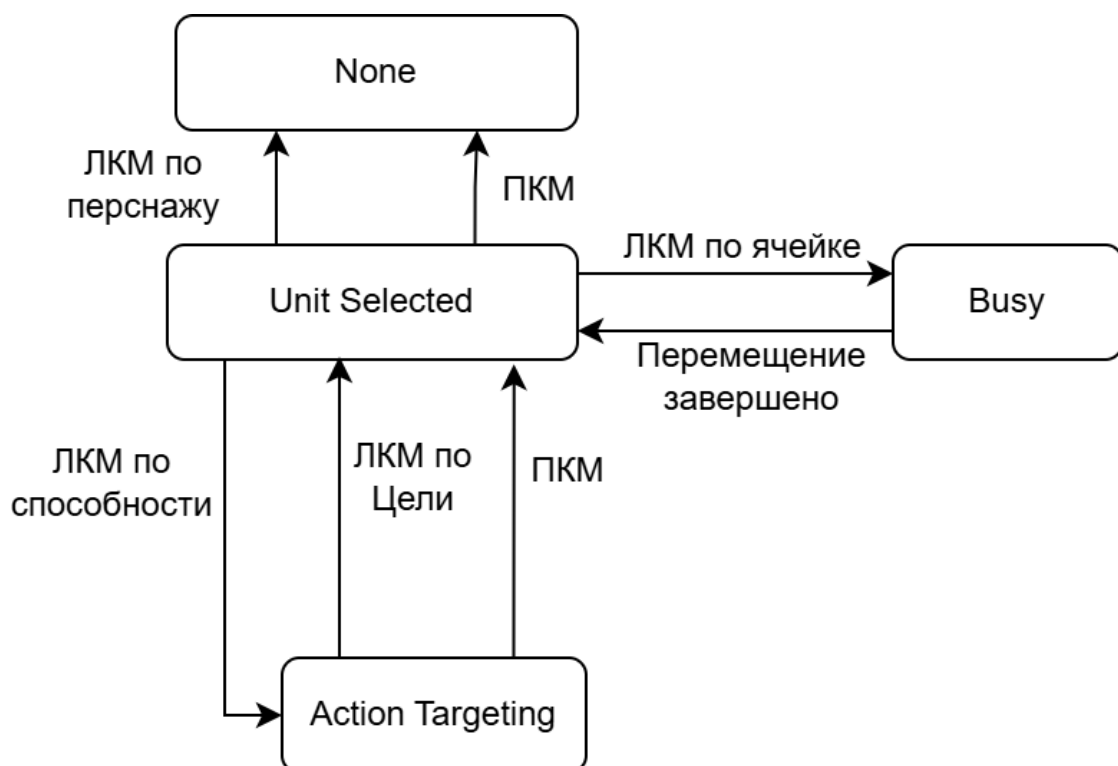


Рисунок 18 – Система состояний игрока

В классе `ATopDownRPGCharacterController` реализованы функции `handleStateChange`, `handleLeftClick`, `handleRightClick`, поведение которых зависит от текущего состояния игрока. Функция `handleStateChange` отвечает за выполнение

действий при входе в новое состояние и при выходе из предыдущего. Функции `handleLeftClick` и `handleRightClick` обрабатывают непосредственно ввод мыши и делегируют выполнение соответствующих логических операций в зависимости от текущего состояния.

При переходе между состояниями система выполняет ряд вспомогательных действий, обеспечивающих визуальную обратную связь и подготовку интерфейса к следующему шагу взаимодействия:

При входе в состояние `UnitSelected`: контроллер получает список доступных способностей выбранного юнита (через компонент `UCharacterAbilitySystem`) и отображает их в пользовательском интерфейсе, активируя панель способностей. Также на выбранного персонажа помещается визуальный маркер (светящийся круг, полупрозрачный силуэт или контур), сигнализирующий игроку, кто именно выбран в данный момент. Маркер цвета, отличного от вражеского (например, зелёный или синий), сопровождается анимацией пульсации для лучшей заметности.

При входе в состояние `ActionTargeting`: контроллер анализирует выбранную способность - извлекает её радиус действия (`Range`), тип допустимых целей (враги, союзники, пустые клетки) и дополнительные ограничения (линия обзора, необходимость свободного пространства). На основе этих параметров система подсвечивает те клетки игрового поля, которые могут быть целями для данной способности. Например, для атакующего заклинания подсвечиваются все враги в радиусе действия, для лечебной способности - союзники с недостатком здоровья, для заклинания создания области - пустые клетки. Подсветка реализована через изменение цвета тайла.

При выходе из любого состояния: контроллер очищает все временные визуальные эффекты, связанные с текущим состоянием. Убирается подсветка клеток, скрывается панель способностей (если она была показана для выбранного юнита), а маркер с персонажа либо полностью удаляется (при переходе в состояние `None`), либо передаётся новому выбранному юниту (при переключении между персонажами). Также сбрасываются временные переменные, хранящие

ссылку на выбранную способность. Это гарантирует, что интерфейс и визуальное отображение всегда соответствуют актуальному игровому контексту.

Дополнительно, для повышения удобства, реализована обработка правой кнопки мыши во всех состояниях: в состоянии `UnitSelected` правая кнопка снимает выделение с персонажа (переход в `None`), в состоянии `ActionTargeting` отменяет выбор способности (возврат в `UnitSelected`), а в состоянии `Busy` (если разрешено) прерывает текущее действие. Такая логика соответствует современным стандартам управления, выявленным при анализе проектов в разделе 1.2.

6.3 Разработка виджетов

Для отображения информации о персонаже и предоставления доступа к его способностям были разработаны два пользовательских виджета, реализованных на базе класса `UUserWidget` - стандартного средства создания интерфейса в `Unreal Engine`. Данный класс предоставляет разработчику гибкие возможности для компоновки визуальных элементов, обработки ввода и привязки данных, что делает его оптимальным выбором для реализации интерфейсных решений любой сложности. Виджеты могут быть сконструированы как визуально в редакторе (`Widget Blueprint`), так и на `C++`, что позволяет сочетать удобство прототипирования с мощностью низкоуровневого кода.

Первый виджет - `UCharacterInfoWidget` - выполняет роль контейнера, агрегирующего и отображающего общую информацию о выбранном персонаже. В его состав входят такие элементы, как имя персонажа, текущее и максимальное значение здоровья (часто в виде числового индикатора и полосы прогресса), портрет, а также иные ключевые характеристики - уровень, запас маны или особых ресурсов, активные статусные эффекты. Такое представление позволяет игроку быстро оценить состояние выбранного юнита без необходимости открывать дополнительные экраны.

Внутри данного виджета располагается элемент `TileView`, предназначенный для динамического отображения списка доступных способностей персонажа [27]. `TileView` представляет собой элемент управления, который автоматически генерирует дочерние виджеты на основе переданного массива данных. Это

особенно удобно, когда набор способностей может меняться в процессе игры - например, при изучении новых заклинаний, смене экипировки или получении временных бонусов. Каждый элемент списка представлен отдельным виджетом *UCharacterActionWidget*, который отвечает за отображение иконки способности, её названия, дальности и, при необходимости, стоимости активации. Использование *TileView* вместо статического набора кнопок делает интерфейс гибким и масштабируемым: добавление новой способности к персонажу не требует перекомпиляции или ручной правки интерфейса - достаточно обновить источник данных. Каркас виджета *UCharacterInfoWidget* представлен на рис.19.

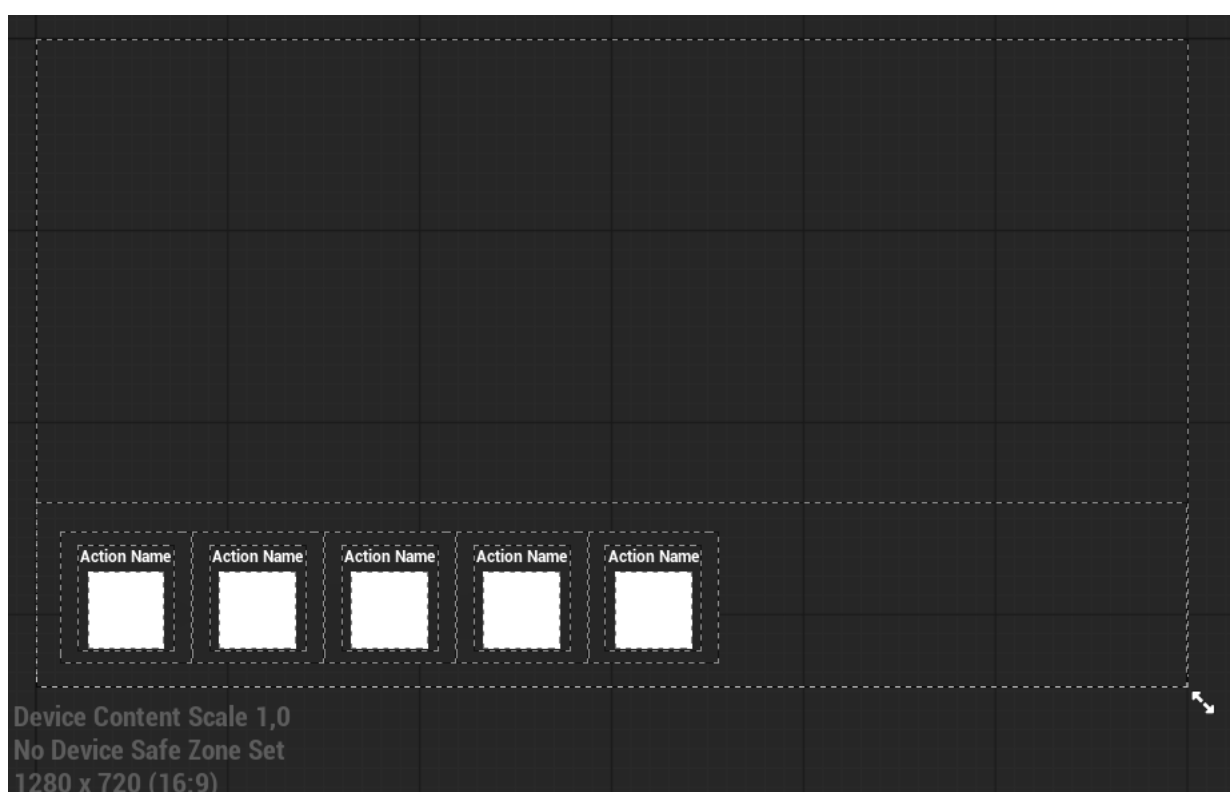


Рисунок 19 – Виджет *UCharacterInfoWidget* в редакторе *UMG*

Второй виджет - *UCharacterActionWidget* - представляет собой элемент списка, отображающий отдельную способность персонажа. Виджет реализует интерфейс *IUserObjectListEntry*, что позволяет ему корректно работать в составе *TileView* и автоматически заполняться данными из привязанного к нему объекта (название способности, иконка, дальность). Структура виджета *UCharacterActionWidget* представлена на рис.20. Полученный HUD представлен на рис. 21.

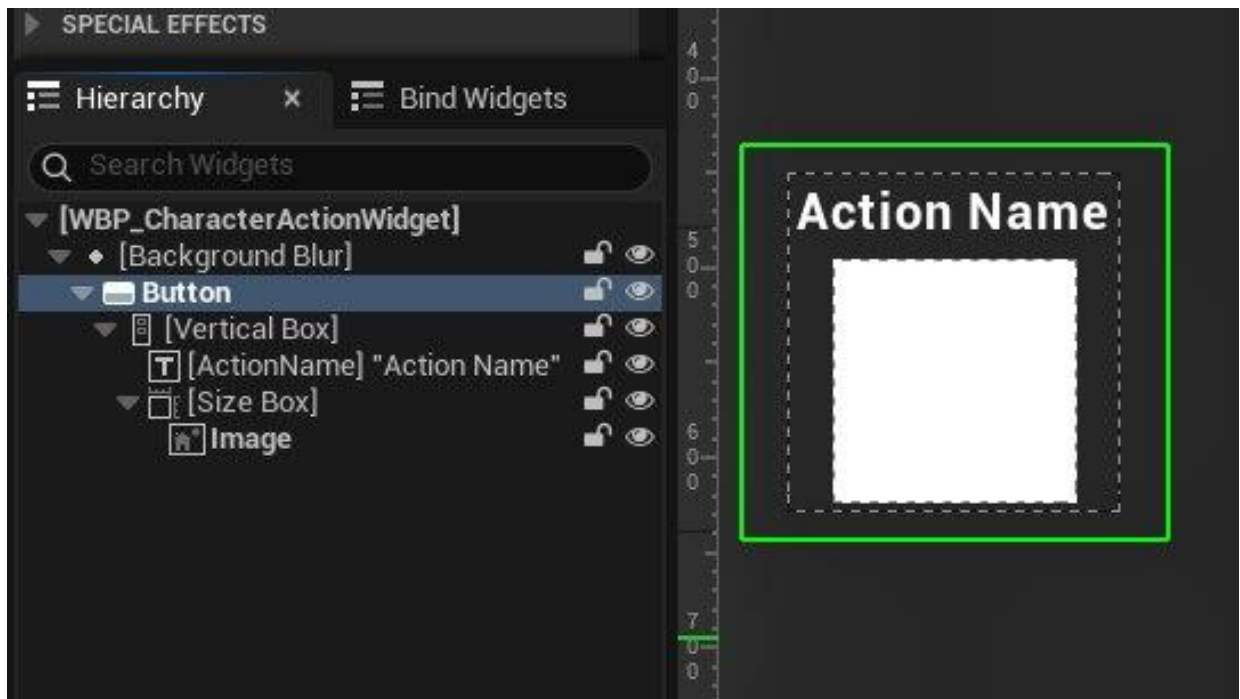


Рисунок 20 – Виджет *UCharacterActionWidget* в редакторе UMG

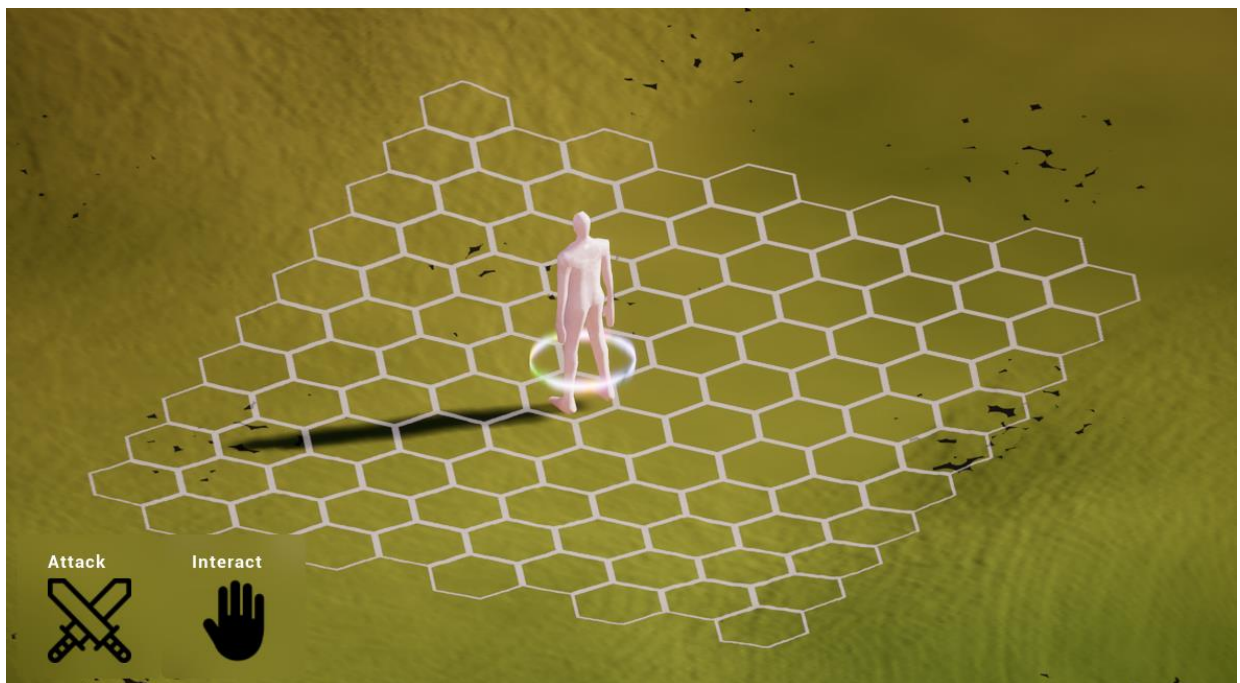


Рисунок 21 – Виджеты, отображаемые в HUD при запуске игры.

6.4 Связывание пользовательского интерфейса с разработанными игровыми механиками

Процесс заполнения интерфейса происходит следующим образом. При клике игрока по персонажу срабатывает логика выбора в контроллере (*TopDownRPGCharacterController*), и система переходит в состоя-

ние *UnitSelected*. В этот момент контроллер обращается к компоненту способностей выбранного персонажа (*UCharacterAbilitySystem*) и получает массив доступных для него способностей (*UCharacterAbility*). Полученный массив передаётся в *TileView*, который автоматически создаёт для каждого элемента массива экземпляр виджета *UCharacterActionWidget* (используя его как шаблон элемента). Каждый созданный виджет получает ссылку на соответствующий объект способности и отображает его визуальное представление - иконку, название, дальность и другие параметры.

Делегаты в Unreal Engine представляют собой механизм вызова методов, позволяющий реализовать паттерн «observer» (Наблюдатель). Делегат объявляет, что некоторое событие произошло, а все подписанные на это событие объекты получают уведомление.

Unreal Engine предоставляет несколько видов делегатов: однозначные (один подписчик), многозначные (множество подписчиков) и динамические (поддерживающие сериализацию и работу в *Blueprints*). В данной работе используются многозначные делегаты, позволяющие при необходимости расширять количество подписчиков без изменения кода.

При создании каждого экземпляра виджета *UCharacterActionWidget* создаётся делегат, инкапсулирующий действие, связанное с данной способностью. Контроллер игрока (*TopDownRPGCharacterController*) подписывается на этот делегат в момент создания виджета.

Когда игрок нажимает на кнопку способности в интерфейсе, делегат срабатывает и передаёт в контроллер объект класса *UCharacterAbility*, соответствующий выбранной способности. Контроллер, получив этот объект, переходит в состояние *ActionTargeting*, запоминая выбранную способность для последующего применения к указанной игроком цели.

С помощью инструментов отладки сетки, разработанных ранее, реализована возможность отображения “зоны поражения” выбранной способности. Ис-

пользуя эвристическую функцию оценки расстояния между клетками и “дальность” конкретной способности, класс сетки подсвечивает клетки, на которые может быть применена способность. Демонстрация представлена на рис. 22.

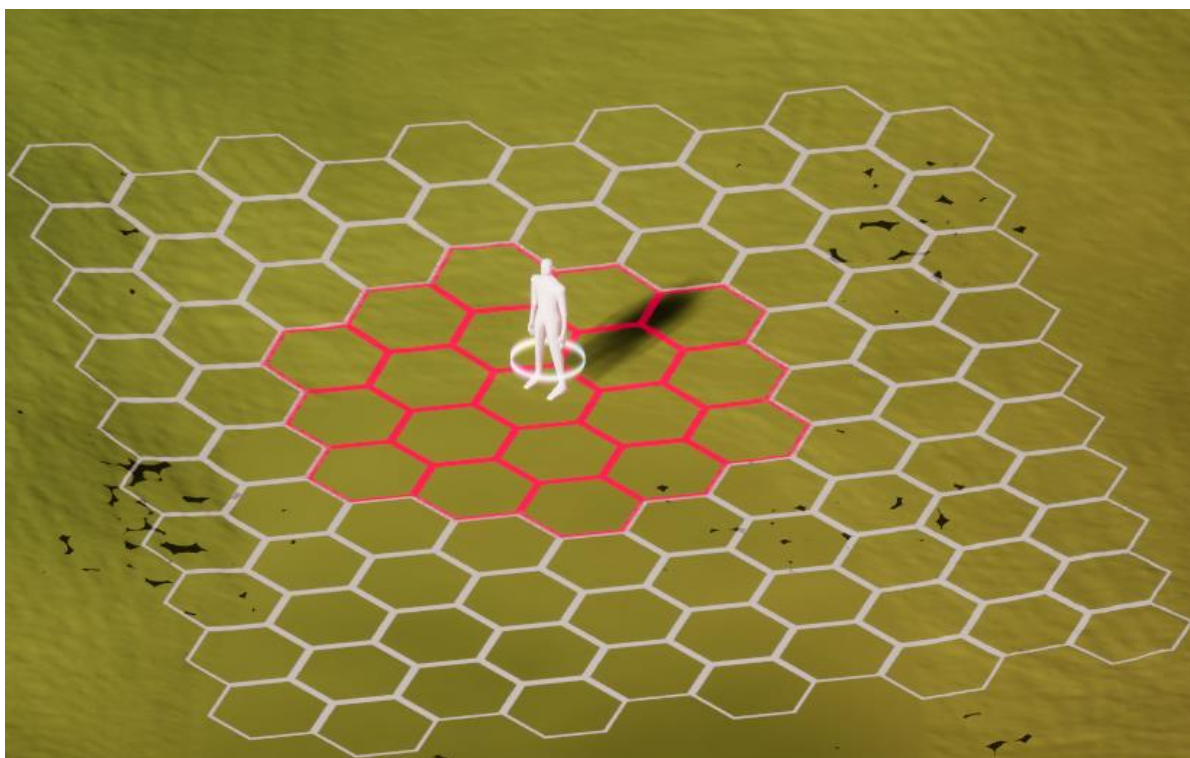


Рисунок 22 – Отображение “зоны поражения” способности.

7 Применение разработанных инструментов

Для проверки работоспособности разработанного инструментария и оценки его применимости в реальных проектах были созданы две тестовые сцены, соответствующие разным типам сеток и разным визуальным пропорциям, выявленным в разделе 1. Первая сцена ориентирована на шестиугольную сетку в стилистике классической игры Fallout (1997), вторая - на квадратную сетку в стилистике современной игры XCOM 2 (2016). Обе сцены были полностью построены с использованием созданных инструментов.

Сцена с шестиугольной сеткой

Была построена тестовая сцена, включающая следующие элементы:

- русло реки, созданное с помощью базовых материалов, предоставляемых Unreal Engine.
- Мост, переброшенный через реку, с корректно отображающейся на нём сеткой;
- скальные образования, размещённые вручную для формирования естественных границ игрового поля.



Рисунок 23 – Основа для игрового поля

Согласно таблице 1, в игре Fallout при разрешении 1280×720 пикселей размер шестиугольной клетки составляет примерно 66×35 пикселей (ширина * высота), а типичный персонаж имеет размер 47×88 пикселей. При пересчёте в условные единицы Unreal Engine (UE) за базовую единицу принят 1 см = 2 UE (стандартное соответствие для персонажей среднего роста). Исходя из этого, в тестовой сцене установлены следующие параметры:

- Расстояние между центрами соседних шестиугольников (диаметр описанной окружности) - 200 UE (что соответствует примерно 100 см или 1 м в игровом пространстве).
- Высота визуального представления клетки (толщина плитки) - 10 UE.
- Высота персонажа - 180 UE (1,8 м), ширина - 90 UE
- Мост через реку - ширина 2 клетки (360 UE), длина 5 клеток (900 UE).

Материал моста настроен так, чтобы проекция навигационной сетки (Runtime Virtual Texture) корректно отображалась на наклонной поверхности.

С помощью инструмента UTileMapToolEditCellsTool на ландшафт была спроецирована шестиугольная сетка размером 25×25 клеток. Клетки, заблокированные ландшафтом (дно реки и скалы), получили статус Disabled. Процесс представлен на рис. 24.

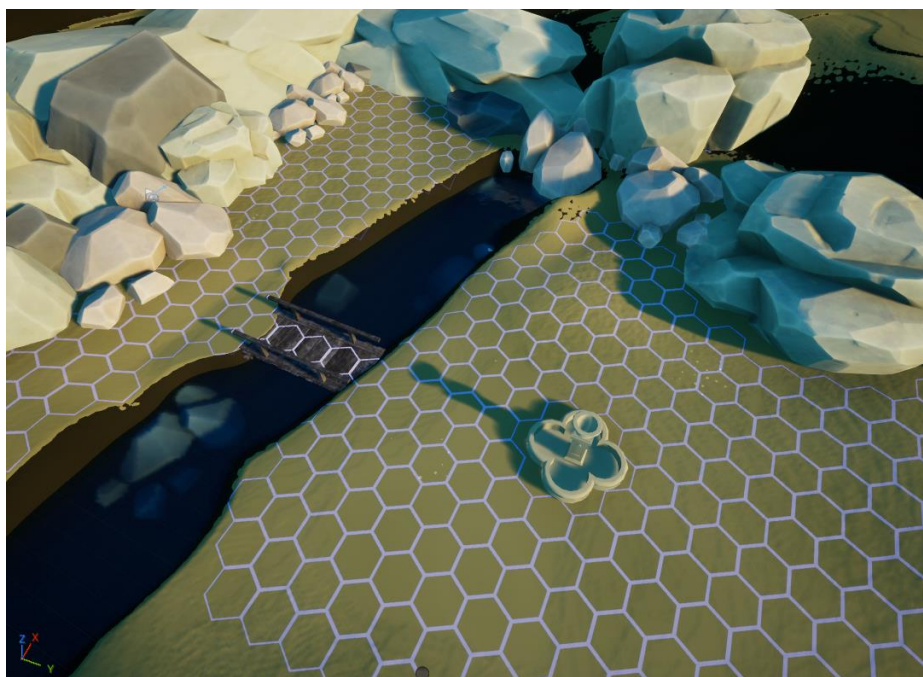


Рисунок 24 – Сетка, нанесенное на игровое поле

Затем с помощью `UTileMapToolPlaceActorsTool` на проходимые клетки были размещены объекты окружения: различные модели камней. Для каждого типа объектов заданы случайные вариации масштаба (от 0,8 до 1,2) и поворота (от 0 до 360 градусов), что позволило избежать эффекта монотонного повторения. На специально выделенных клетках (со статусом `Empty`) размещён управляемый персонаж (класс `ATileContentCharacter`) с шестиугольной сеткой навигации. На рис. 25 и 26 представлен вид итоговой сцены с шестиугольной сеткой в режиме игры.

Особенности реализации. Для шестиугольной сетки в классе `ATilemapHex` переопределена функция `Heuristic` - эвристика использует расстояние на шестиугольной сетке (координаты куба). Поиск пути работает корректно, учитывая непроходимые клетки и клетки, занятые объектами.

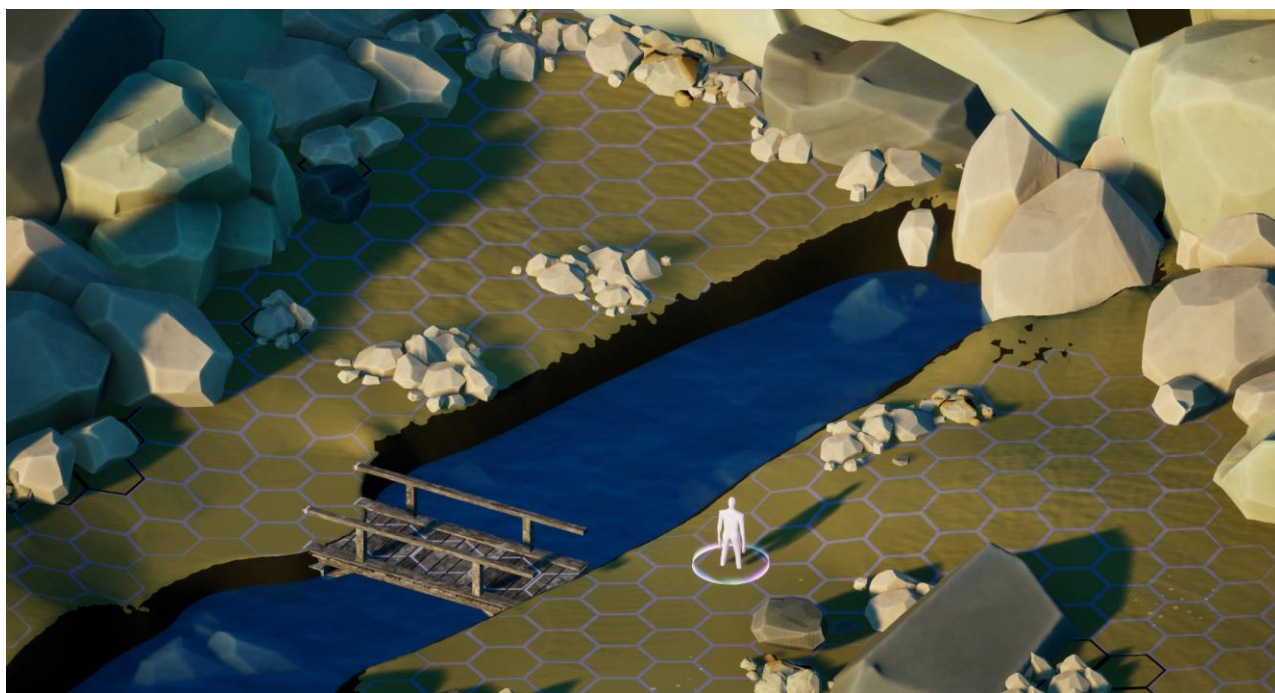


Рисунок 25 – Готовая сцена в стиле Fallout

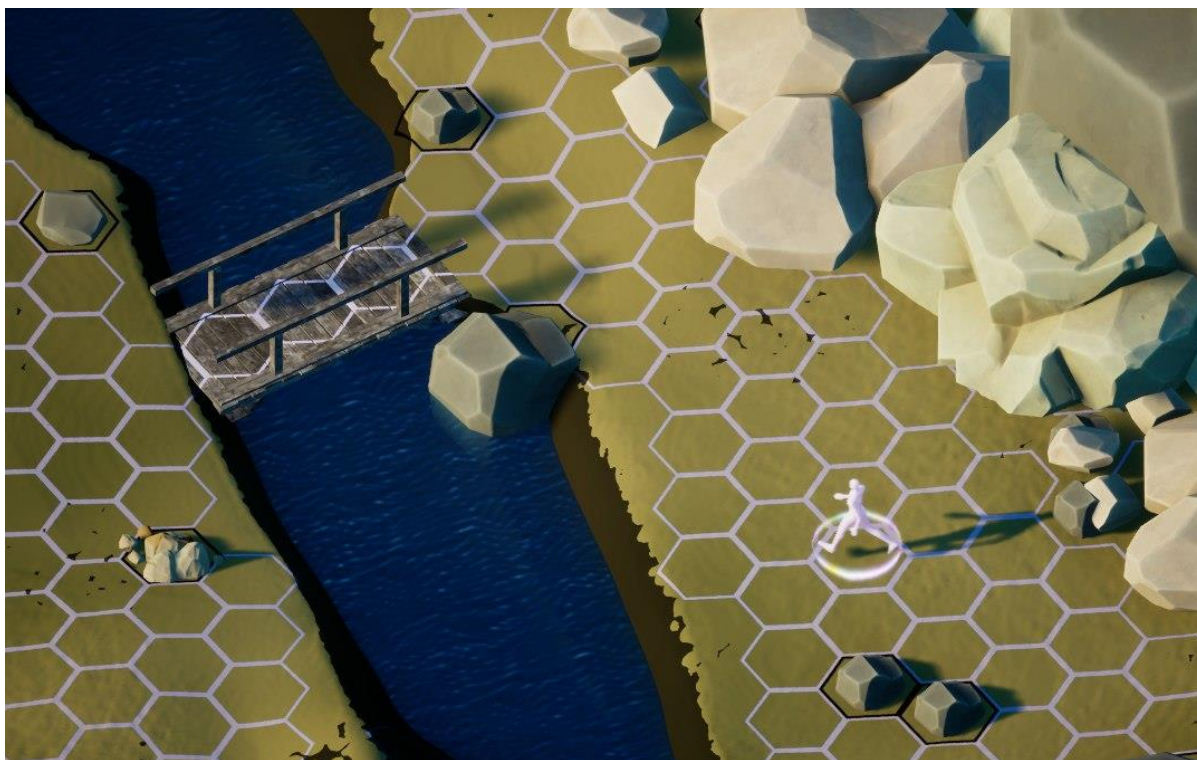


Рисунок 26 – Готовая сцена в стиле Fallout

Сцена с квадратной сеткой (на основе XCOM 2)

По данным таблицы 1, в XCOM 2 при разрешении 1280×720 размер квадратной клетки в центре экрана составляет примерно 90×64 пикселя, персонаж - 45×90 пикселей. Для трёхмерной сцены приняты следующие значения:

Размер стороны квадратной клетки - 180 UE (ширина и глубина). Это немного меньше, чем в шестиугольной сцене, что соответствует более плотной сетке современной тактической игры.

Высота персонажа - 170 UE (1,7 м), ширина - 80 UE. Соотношение высоты персонажа к размеру клетки ($170/180 \approx 0,94$) обеспечивает комфортное зрительное восприятие: персонаж не перекрывает соседние клетки.

Для демонстрации разработанного шейдера, позволяющего делать визуальные препятствия полупрозрачными, в программе Revit была создана 3д модель башни. Модель была разделена на два материала, что нужно для возможности проецирования сетки на пол постройки. Модель и её разделение на материалы представлены на рис.27.

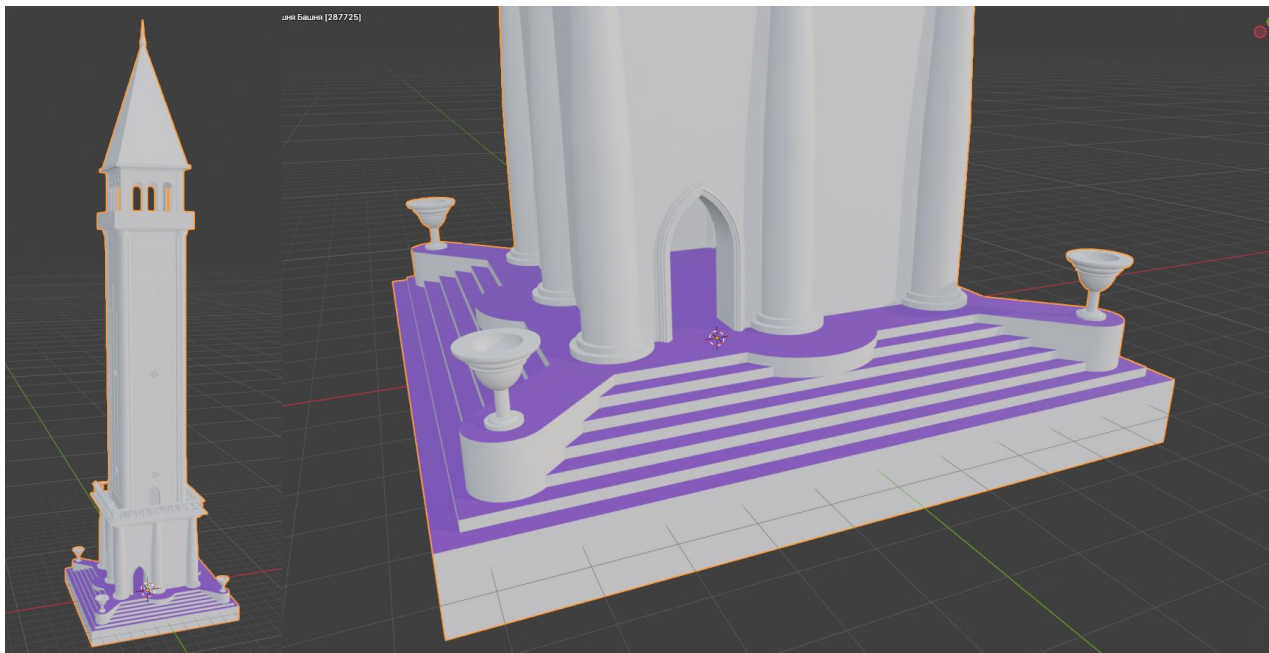


Рисунок 27 - 3Д модель, разделенная на два материала

На простом ландшафте создана квадратная сетка размером 21×21 клеток. В центр композиции помещена 3д модель башни, к верхней части башни применен полупрозрачный материал.

Полученная сцена представлена на рис. 28. Она демонстрирует возможность использования разработанных инструментов для создания сцен внутри закрытых помещений.

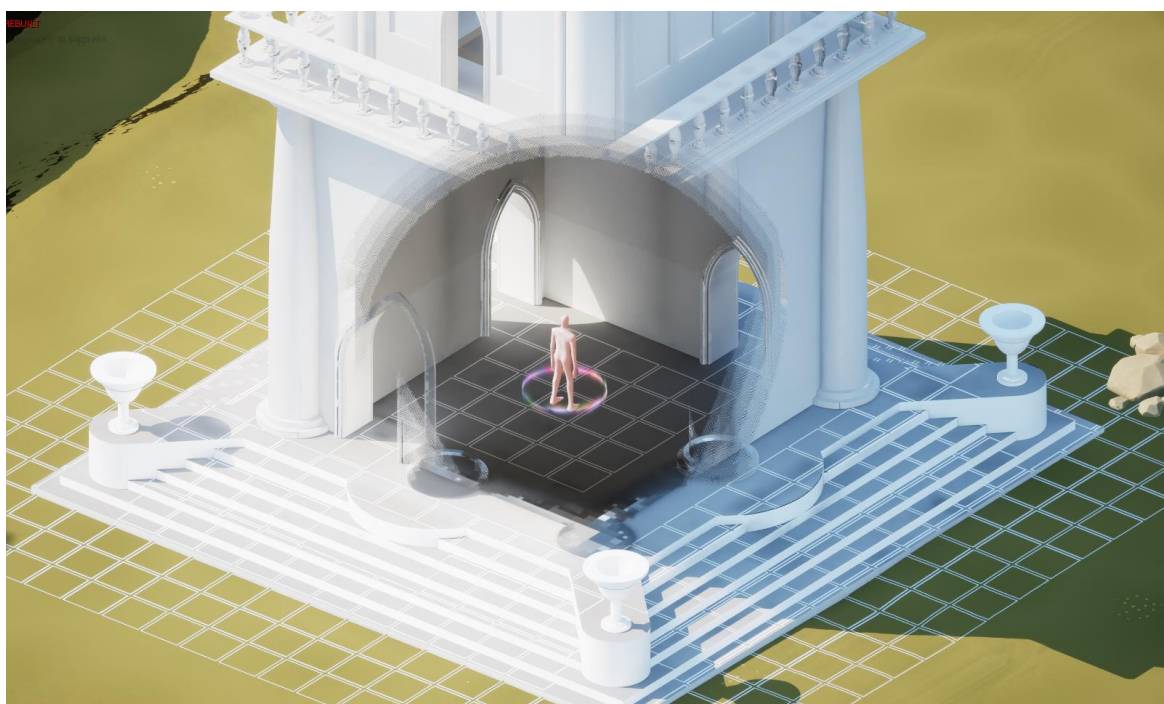


Рисунок 28 – Использование сетки на игровой карте с закрытым помещением

8 Анализ результатов

Разработанный плагин спроектирован с учётом принципов расширяемости и модульности, что позволяет масштабировать функциональность в различных направлениях. Все ключевые классы и системы открыты для наследования и переопределения, предоставляя разработчику возможность адаптировать плагин под нужды конкретного проекта без изменения исходного кода. Основные направления масштабирования проекта представлены в таблице 2.

Таблица 2

Основные направления масштабирования

Компонент	Возможности расширения
Сетка (Tilemap)	Базовый класс ATilemapBase позволяет реализовать альтернативные формы сетки - например, прямоугольную, изометрическую или сетку с нестандартными размерами ячеек. Достаточно создать новый класс, наследующий от ATilemapBase, и переопределить методы генерации и навигации.
Навигация	Алгоритм поиска пути может быть заменён на альтернативный (Dijkstra, волновой алгоритм) путём переопределения метода Navigate в классе сетки. Это позволяет выбирать наиболее подходящий алгоритм в зависимости от формы игровых уровней и требований к производительности.
Персонажи	Базовый класс ATileContentCharacter может быть расширен для создания различных типов юнитов - игровых персонажей, врагов, NPC, союзников. Разработчик может добавлять новые компоненты, переопределять поведение и настраивать параметры под конкретные задачи.
Способности	Система способностей построена на наследовании от UCharacterAbility. Добавление новой способности (огненный шар, исцеление, телепортация, призыв существа)

	требует лишь создания нового класса, наследующего от UCharacterAbility, и переопределения метода Execute. Благодаря спецификатору BlueprintNativeEvent новые способности можно создавать как на C++, так и в Blueprints без написания кода.
Пользовательский интерфейс	Виджеты UCharacterInfoWidget и UCharacterActionWidget могут быть расширены или полностью заменены собственными реализациями. Разработчик может добавить новые элементы интерфейса: панель инвентаря, журнал квестов, диалоговое окно
Система состояний	Набор EPlayerControllerState и логика переходов могут быть дополнены новыми состояниями: InventoryOpened, DialogueMode, BuildingMode путём расширения контроллера и добавления соответствующих обработчиков.

ЗАКЛЮЧЕНИЕ

В ходе выполнения работы была достигнута поставленная цель - разработка плагина для Unreal Engine 5, расширяющего функционал движка и предоставляющего инструменты для создания проектов в жанре Top-Down RPG. Для достижения цели были решены следующие задачи:

1. Проведён обзор и анализ инструментов, предлагаемых движком Unreal Engine 5 для создания пользовательских режимов редакторов, обработки ввода, и навигации;
2. Проведён обзор коммерчески успешных проектов разных эпох - с точки зрения визуального оформления, а также эволюции систем управления и взаимодействия с игровым миром. На основе анализа было принято решение о реализации контекстно-зависимого управления через левую кнопку мыши по аналогии с современными представителями жанра;
3. Разработаны инструменты разработчика для создания tilemaps - игрового поля, разделённого на клетки, с поддержкой квадратной и шестиугольной сетки, а также возможность интерактивного взаимодействия с клетками (выбор, подсветка, изменение состояния);
4. Разработан игровой персонаж, имеющий возможность перемещаться по игровому полю, а также имеющий систему способностей, которые можно применять на игровое поле;
5. Разработан пользовательский интерфейс: реализована система управления камерой (панорамирование, масштабирование), а также механизмы взаимодействия с игровым миром (выбор персонажа, выбор интерактивных объектов, навигация по полю) с использованием левой кнопки мыши и выбором действий персонажа с помощью визуального интерфейса.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. RPG Maker // Kadokawa Games URL: <https://www.rpgmakerweb.com/>. Дата обращения: 17.02.2026.
2. Turn Based Strategy Framework // Crooked Head URL: <https://assetstore.unity.com/packages/templates/systems/turn-based-strategy-framework-50282>. Дата обращения: 18.02.2026.
3. Unity Tilemap System // Unity Documentation URL: <https://docs.unity.com/en-us>. Дата обращения: 19.02.2026.
4. Advanced Turn Based Tile Toolkit Plugin // Knut Øverbye URL: <https://unrealmonster.com/advanced-turn-based-tile-toolkit-5-0-5-3/>. Дата обращения: 20.02.2026.
5. Unreal Engine 4 Documentation // Unreal Engine Documentation URL: <https://docs.unrealengine.com/>. Дата обращения: 15.02.2026.
6. Custom Editor Mode Customization (UEdMode) - Additional Discussions // Epic Developer Community Forums. URL: <https://forums.unrealengine.com/t/custom-editor-mode-customization-uedmode>. Дата обращения: 03.06.2026.
7. Interactive Tools Framework in Unreal Engine // Epic Developer Community. URL: <https://dev.epicgames.com/documentation/en-us/unreal-engine/interactive-tools-framework-in-unreal-engine>. Дата обращения: 05.04.2026.
8. X-Plore: Turn-Based Tactics Toolkit // GitHub. URL: <https://github.com/JameelHossain/X-Plore>. Дата обращения: 28.04.2026.
9. Custom Editor Mode Customization (UEdMode) // Epic Developer Community Forums. URL: <https://forums.unrealengine.com/t/custom-editor-mode-customization-uedmode>. Дата обращения: 15.03.2026.
10. Unreal Engine Modules // Unreal Engine 5.2 Documentation. Epic Developer Community. URL: <https://dev.epicgames.com/documentation/unreal-engine/unreal-engine-modules>. Дата обращения: 13.05.2026.

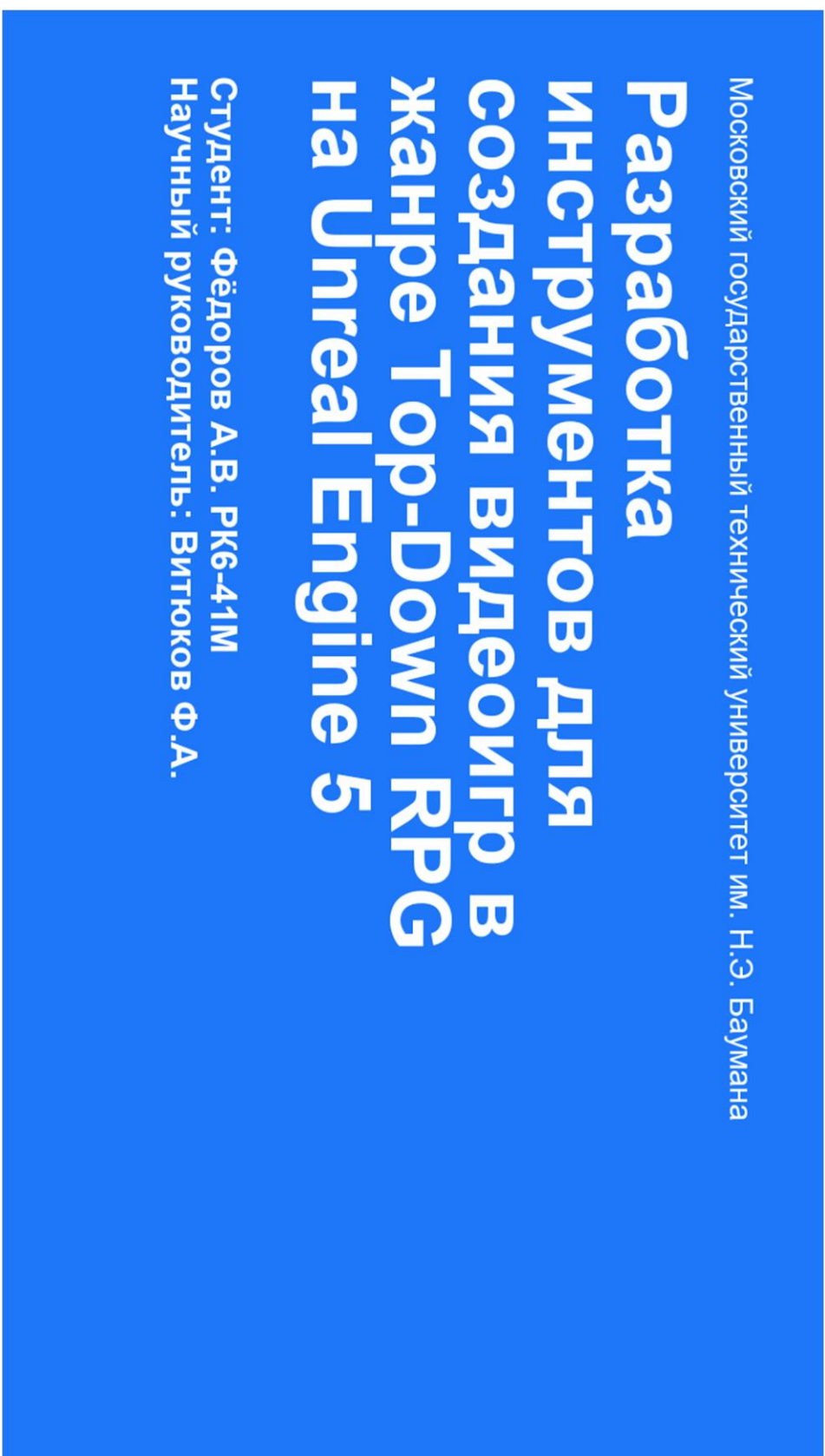
- 11.Plugins in Unreal Engine // Unreal Engine 5.2 Documentation. Epic Developer Community. URL: <https://dev.epicgames.com/documentation/unreal-engine/plugins-in-unreal-engine>. Дата обращения: 12.05.2026.
- 12.Optimizing Real-Time Path Planning for NPC Navigation Leveraging CentA* Algorithm // GitHub. URL: <https://github.com/vinit714/Optimizing-Real-Time-Path-Planning-for-NPC-Navigation-Leveraging-CentA--Algorithm>. Дата обращения: 30.05.2026.
- 13.UMG Widget Interaction Components in Unreal Engine // Epic Developer Community. URL: <https://dev.epicgames.com/documentation/en-us/unreal-engine/umg-widget-interaction-components-in-unreal-engine>. Дата обращения: 20.03.2026.
- 14.Runtime Virtual Texturing in Unreal Engine // Epic Developer Community. URL: <https://dev.epicgames.com/documentation/unreal-engine/runtime-virtual-texturing-in-unreal-engine>. Дата обращения: 05.05.2026.
- 15.UInstancedStaticMeshComponent // Unreal Engine 5.0 Documentation. Epic Developer Community. URL: <https://dev.epicgames.com/documentation/unreal-engine/API/Runtime/Engine/Components/UInstancedStaticMeshComponent>. Дата обращения: 06.05.2026.
- 16.Research on UE5 Pathfinding Based on Improved A* Algorithm // IEEE Xplore. URL: <https://ieeexplore.ieee.org/document/12345678>. Дата обращения: 15.04.2026.
- 17.Instanced Static Mesh Component in Unreal Engine // Unreal Engine 5.4 Documentation. Epic Developer Community. URL: <https://dev.epicgames.com/documentation/unreal-engine/instanced-static-mesh-component-in-unreal-engine>. Дата обращения: 07.05.2026.
- 18.Editor Subsystems in Unreal Engine // Epic Developer Community. URL: <https://dev.epicgames.com/documentation/en-us/unreal-engine/editor-subsystems-in-unreal-engine>. Дата обращения: 02.04.2026.

- 19.Procedural Content Generation Overview // Unreal Engine 5.7 Documentation. Epic Developer Community. URL: <https://dev.epicgames.com/documentation/unreal-engine/procedural-content-generation-overview>. Дата обращения: 25.05.2026.
- 20.Gameplay Ability System for Unreal Engine // Unreal Engine 5.0 Documentation. Epic Developer Community. URL: <https://dev.epicgames.com/documentation/unreal-engine/gameplay-ability-system-for-unreal-engine>. Дата обращения: 15.05.2026.
- 21.Gameplay Ability System for Unreal Engine // Epic Developer Community. URL: <https://dev.epicgames.com/documentation/en-us/unreal-engine/gameplay-ability-system-for-unreal-engine>. Дата обращения: 10.03.2026.
- 22.Gameplay Attributes and Attribute Sets for the Gameplay Ability System // Unreal Engine 5.0 Documentation. Epic Developer Community. URL: <https://dev.epicgames.com/documentation/unreal-engine/gameplay-attributes-and-attribute-sets-for-the-gameplay-ability-system>. Дата обращения: 17.05.2026.
- 23.Gameplay Attributes and Gameplay Effects for the Gameplay Ability System // Unreal Engine 5.1 Documentation. Epic Developer Community. URL: <https://dev.epicgames.com/documentation/unreal-engine/gameplay-attributes-and-gameplay-effects-for-the-gameplay-ability-system>. Дата обращения: 18.05.2026.
- 24.Abilities in Lyra for Unreal Engine // Unreal Engine 5.5 Documentation. Epic Developer Community. URL: <https://dev.epicgames.com/documentation/unreal-engine/abilities-in-lyra-in-unreal-engine>. Дата обращения: 22.05.2026.
- 25.Designer 04 Player HUD in Unreal Engine // Epic Developer Community. URL: <https://dev.epicgames.com/documentation/en-us/unreal-engine/designer-04-player-hud-in-unreal-engine>. Дата обращения: 25.03.2026.
- 26.Lyra Sample Game in Unreal Engine // Unreal Engine 5.7 Documentation. Epic Developer Community. URL: <https://dev.epicgames.com/documentation/unreal-engine/lyra-sample-game-in-unreal-engine>. Дата обращения: 20.05.2026.

27. Creating User Interfaces with UMG and Slate in Unreal Engine // Epic Developer Community. URL: <https://dev.epicgames.com/documentation/en-us/unreal-engine/creating-user-interfaces-with-umg-and-slate-in-unreal-engine>. Дата обращения: 20.04.2026.

ПРИЛОЖЕНИЕ А. Графическое представление выпускной квалификационной работы

В графическую часть выпускной квалификационной работы входят 10 слайдов:



Постановка задачи

Цель работы: расширить функционал 3д движка Unreal Engine 5 инструментами для создания видеоигр в жанре Top-Down RPG.

Задачи:

1. Изучить успешные проекты жанра;
2. Разработать инструменты для создания и настройки игрового поля, разбитого на тайлы;
3. Разработать игрового персонажа, взаимодействующего с игровым полем;
4. Разработать пользовательский интерфейс для возможности управления персонажами.

Актуальность работы

- Стремительное развитие российской индустрии видеоигр;
- Мировая популярность жанра Top-Down RPG;
- Рассматриваемые технологии используются в подавляющем большинстве современных проектов жанра.

Актуальность работы



Рисунок 1 – Современные проекты жанра Тор-Доун
RPG

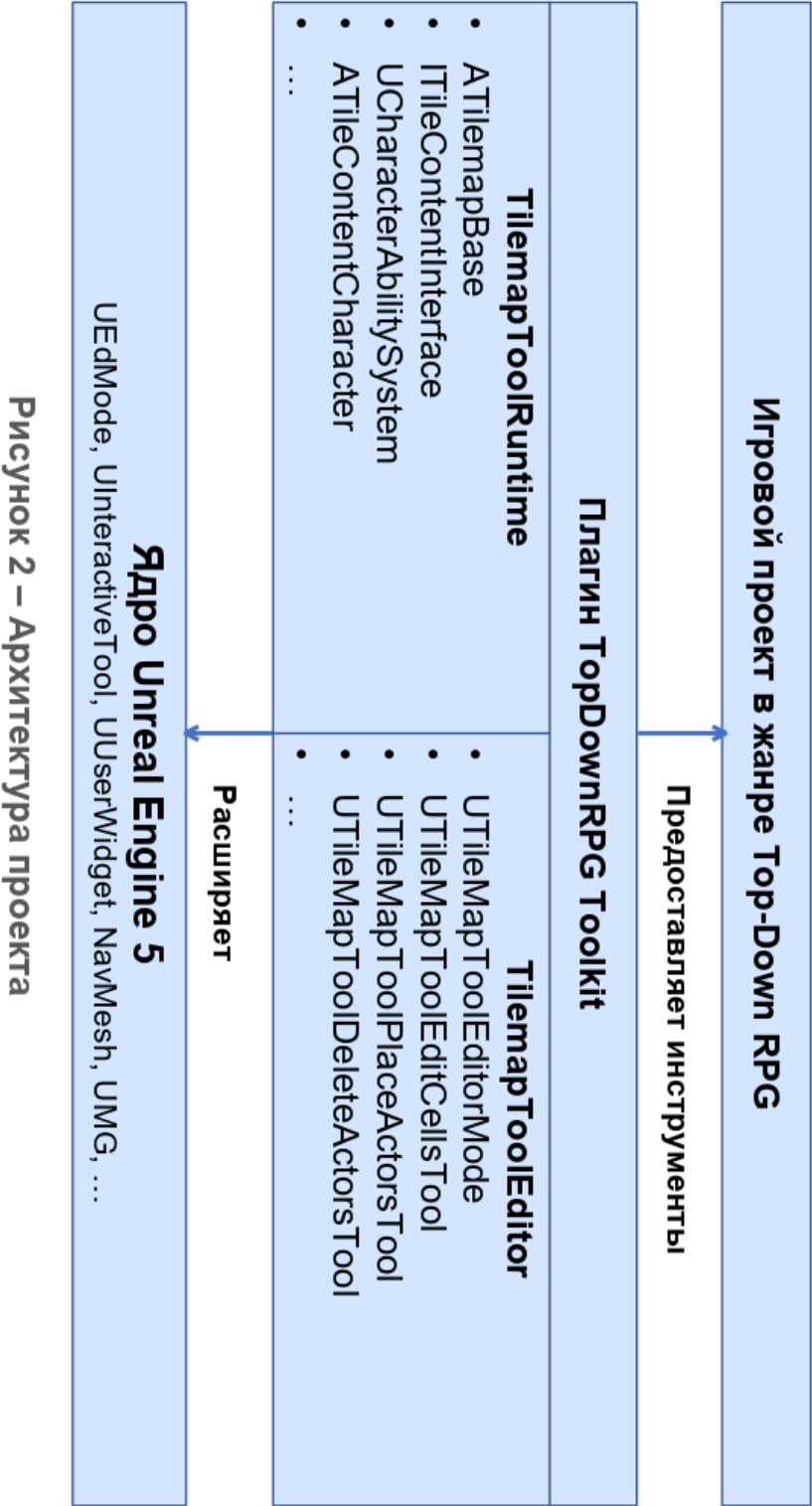
Проект	Разработчик	Число проданных копий
Baldur's Gate 3 (2023)	Larian Studios (Бельгия)	>20 000 000
Diablo 3 (2012)	Blizzard Entertainment (США)	>30 000 000
Pathfinder: Kingmaker	Owlcat Games (Россия)	~2 000 000

Таблица 1 – Продажи представленных проектов

Основные понятия

- **Top-Down RPG** – ролевая игра с видом сверху;
- **Tilemap** – игровое поле, разбитое на дискретные ячейки (тайлы);
- **Instancing (дублирование геометрии)** – подход, позволяющий отрисовывать множество копий одного и того же 3d-объекта за один проход;

Структура проекта



Разработка игрового поля

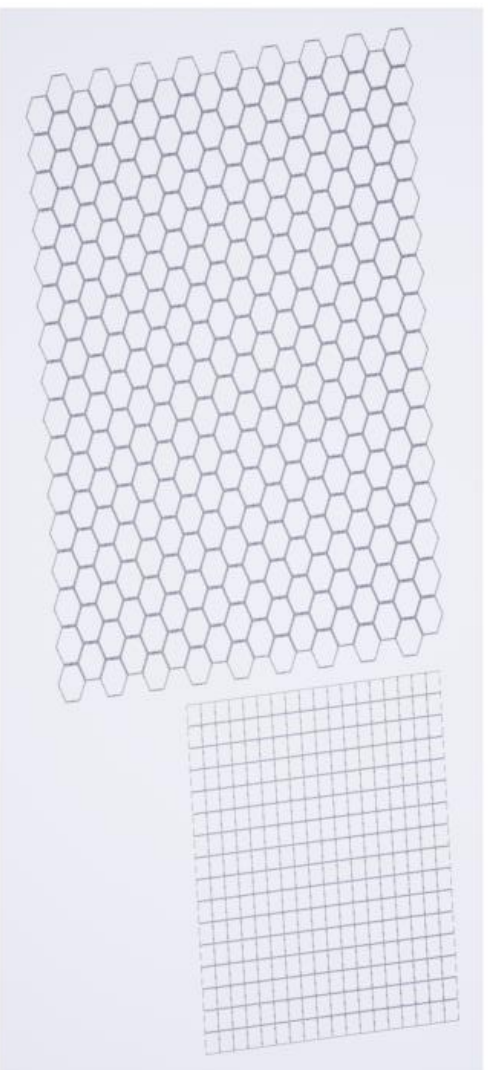


Рисунок 3 – Шестиугольная и квадратная сетки

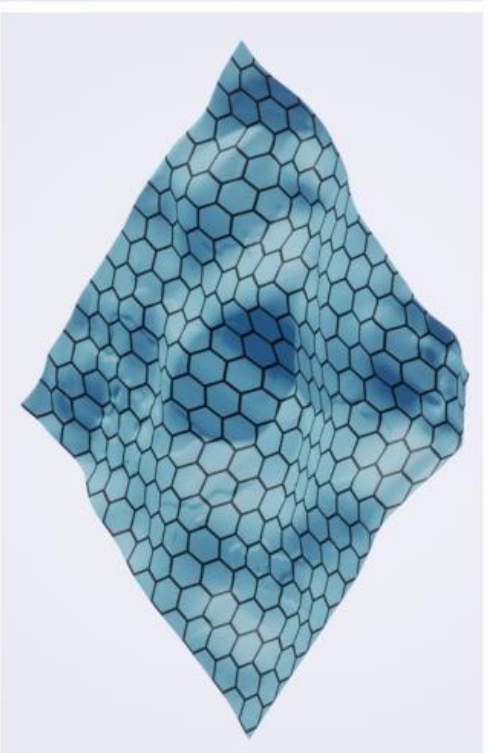


Рисунок 4 – Сетка, спроецированная на игровое поле произвольной формы

Разработка нового режима редактора Unreal Engine 5

Функционал редактора:

- Инструмент для размещения игровых объектов на поле;
- Инструмент для удаления игровых объектов с поля;
- Инструмент для изменения статуса индивидуальных тайлов.

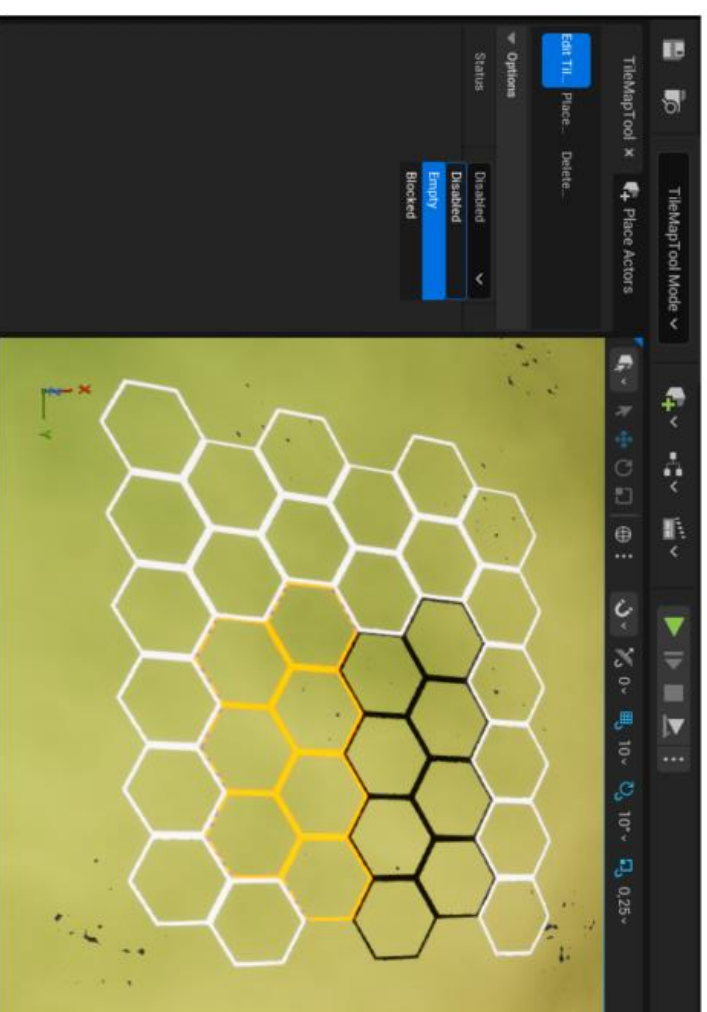


Рисунок 5 – Графический интерфейс разработанного режима редактора

Разработка игрового персонажа: навигация

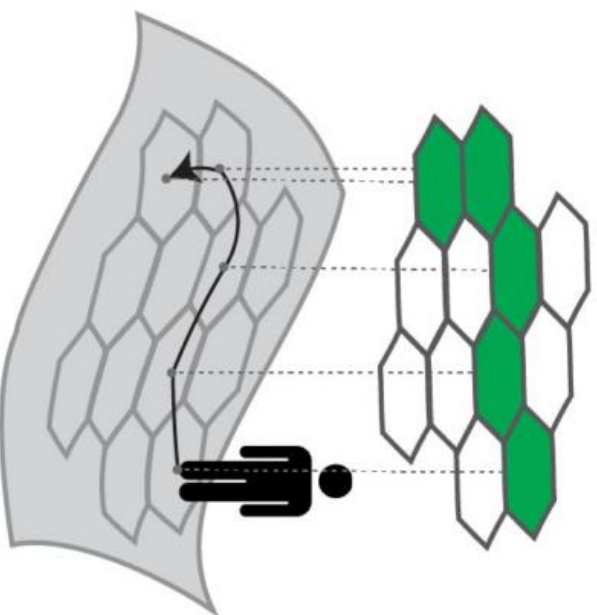


Рисунок 6 – Схема навигации персонажа по игровому полю

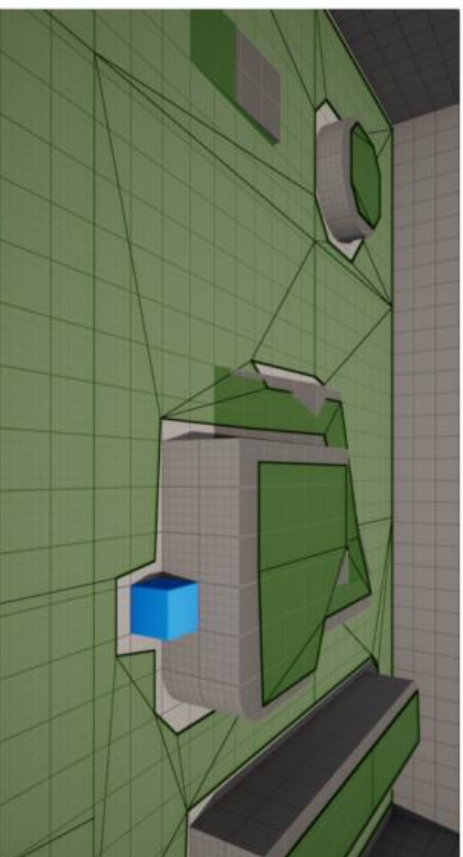


Рисунок 7 – Визуализация NavMesh

Разработка пользовательского интерфейса: Виджеты



Рисунок 8 – Виджет
индивидуальной способности
персонажа

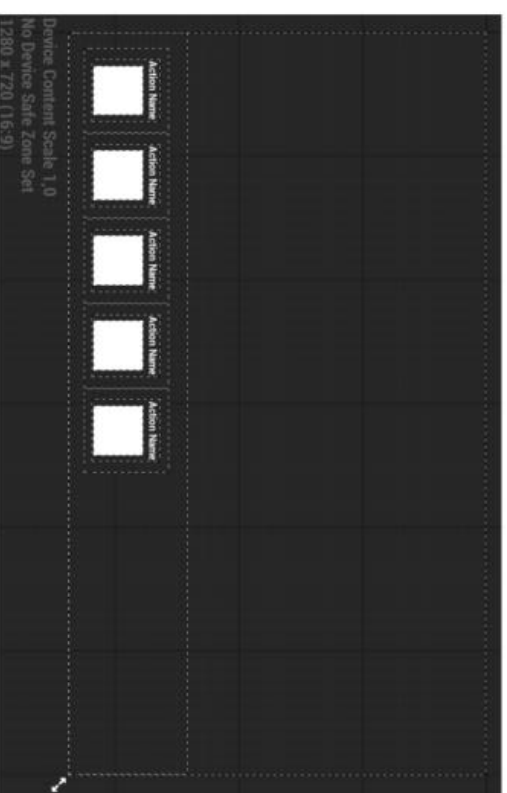


Рисунок 9 – Виджет полного набора
способностей персонажа

Разработка пользовательского интерфейса: интеграция с разработанными системами

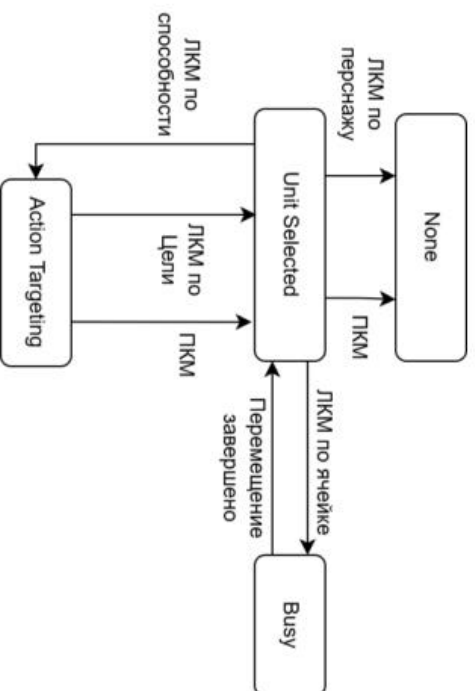


Рисунок 10 – Система состояний игрока

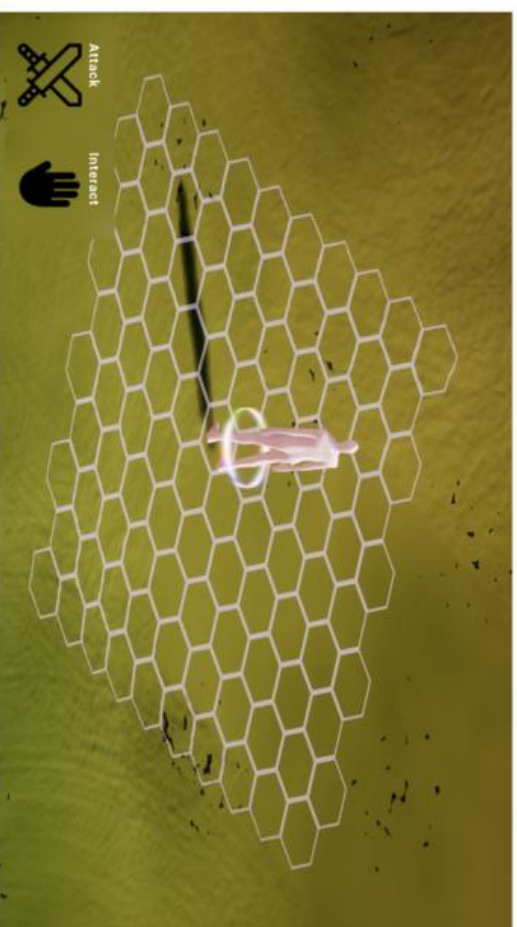


Рисунок 11 – Графический интерфейс,
отображаемый при выборе персонажа

Апробация разработанных инструментов

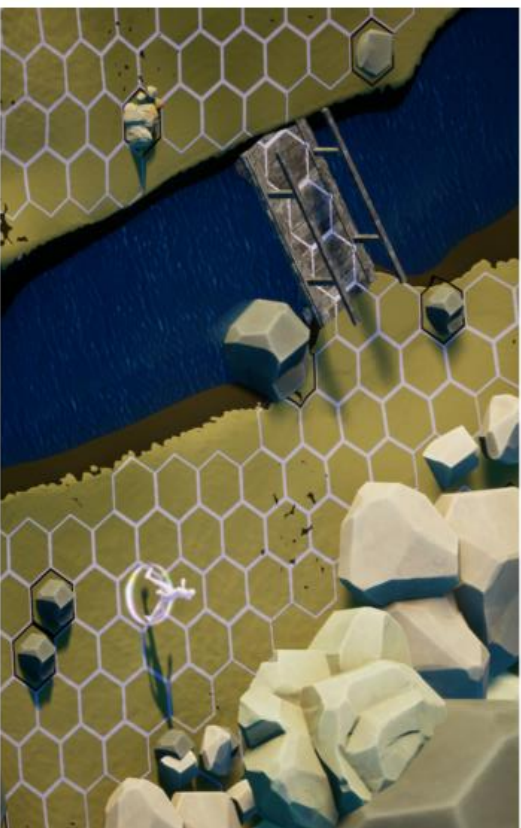


Рисунок 12 – Сцена, демонстрирующая
использование шестиугольной сетки

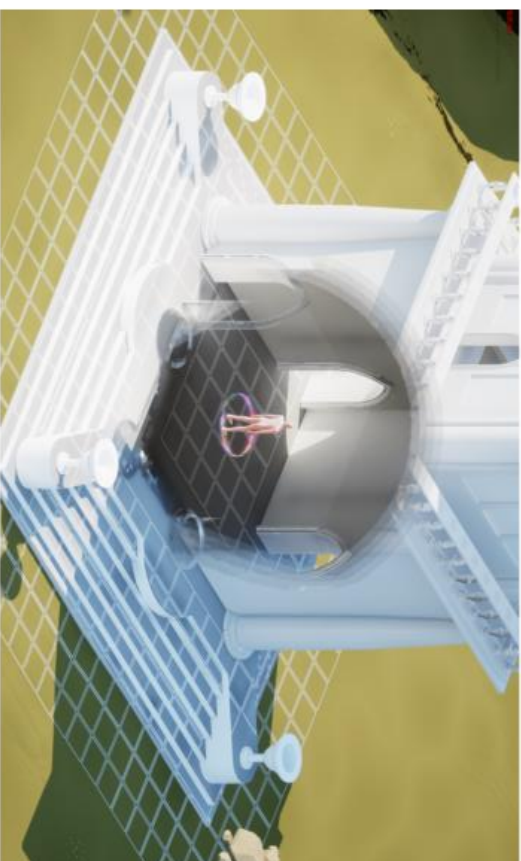


Рисунок 13 – Сцена, демонстрирующая
использование квадратной сетки

Возможности для расширения

- Сетки произвольной формы;
- Альтернативные алгоритмы навигации по карте;
- Создание различных типов персонажей: врагов, союзников, NPC с искусственным интеллектом;
- Новые состояния игрока, например для поддержки системы инвентаря или системы диалогов.

Заключение

В результате работы выполнены следующие задачи:

1. Изучены успешные проекты жанра;
2. Разработаны инструменты для создания и настройки игрового поля, разбитого на тайлы;
3. Разработан игрового персонаж, имеющий возможность перемещаться по игровому полю и использовать способности;
4. Разработан пользовательский интерфейс, позволяющий управлять игровыми персонажами с помощью кнопок мыши.